

Unit 6. Support Vector Machines, Kernels, and Neural Networks

Business Data Analytics: Introductory Machine Learning

From hyperplanes to margins

Logistic regression in Unit 3 placed a hyperplane to maximize the data likelihood. The support vector machine places a hyperplane to maximize the geometric distance from the closest training point. The two criteria differ; the second one delivers a constrained convex problem whose dual exposes which training points actually shape the decision boundary, and turns the inner product $\mathbf{x}_i^\top \mathbf{x}_j$ into a slot a kernel can occupy.

1 Constrained Convex Optimization

The convex analysis chapter of Unit 1 (§1.9) treated unconstrained minimization: gradient zero, Hessian PSD, done. The SVM problems in this unit are constrained: minimize a convex objective subject to one inequality per training point. Three additions to the Unit 1 toolkit handle the constrained case: the Lagrangian, weak and strong duality, and the KKT conditions.

1.1 Constrained problems in standard form

Definition 6.1 (Constrained optimization problem). A **constrained optimization problem** in standard form is

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^n} \quad & f_0(\mathbf{x}) \\ \text{subject to} \quad & f_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, m, \\ & h_j(\mathbf{x}) = 0, \quad j = 1, \dots, p. \end{aligned}$$

The function f_0 is the **objective**; $f_1, \dots, f_m$ are the **inequality constraint functions**; $h_1, \dots, h_p$ are the **equality constraint functions**. The **feasible set** is

$$\mathcal{D} = \{ \mathbf{x} \in \mathbb{R}^n : f_i(\mathbf{x}) \leq 0 \text{ for all } i, \ h_j(\mathbf{x}) = 0 \text{ for all } j \}.$$

The **optimal value** is $p^* = \inf\{f_0(\mathbf{x}) : \mathbf{x} \in \mathcal{D}\}$, with the conventions $p^* = +\infty$ when $\mathcal{D} = \emptyset$ (the problem is **infeasible**) and $p^* = -\infty$ when f_0 is unbounded below on $\mathcal{D}$. A point $\mathbf{x}^* \in \mathcal{D}$ achieving $f_0(\mathbf{x}^*) = p^*$ is an **optimal point**.

The problem is a **convex optimization problem** when $f_0, f_1, \dots, f_m$ are convex on $\mathbb{R}^n$ and each

h_j is affine, $h_j(\mathbf{x}) = \mathbf{a}_j^\top \mathbf{x} - b_j$. Affine equalities define a translated subspace; convex sublevel sets $\{f_i \leq 0\}$ are convex (Unit 1 §1.9.8, epigraph characterization); their intersection $\mathcal{D}$ is therefore convex. Both Unit 1's unconstrained problems ($m = p = 0$) and the SVM formulations developed below fit this template.

Definition 6.2 (Slater's condition). A convex problem in the form of Definition 6.1 satisfies **Slater's condition** if there exists $\tilde{\mathbf{x}} \in \mathcal{D}$ with

$$f_i(\tilde{\mathbf{x}}) < 0, \quad i = 1, \dots, m,$$

i.e., a feasible point at which every nonlinear inequality constraint holds strictly. When some f_i is itself affine, the strict inequality is relaxed to $f_i(\tilde{\mathbf{x}}) \leq 0$ at $\tilde{\mathbf{x}}$ for those affine constraints; strict feasibility is required only for the genuinely nonlinear convex inequalities.

Slater's condition is the standard **constraint qualification** for convex problems. It rules out the degenerate case of a convex inequality that is "tight everywhere" on the feasible set, a case in which the primal and dual optimal values (defined below) can differ. For every primal SVM problem written down later in this unit, Slater's condition holds.

1.2 Lagrangian and dual function

The constrained problem of Definition 6.1 becomes more tractable after merging the constraints into the objective with one multiplier per constraint.

Definition 6.3 (Lagrangian). The **Lagrangian** associated with the constrained problem of Definition 6.1 is the function $L : \mathbb{R}^n \times \mathbb{R}^m \times \mathbb{R}^p \rightarrow \mathbb{R}$,

$$L(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\nu}) = f_0(\mathbf{x}) + \sum_{i=1}^m \lambda_i f_i(\mathbf{x}) + \sum_{j=1}^p \nu_j h_j(\mathbf{x}).$$

The scalars $\lambda_1, \dots, \lambda_m$ are the **Lagrange multipliers** for the inequality constraints; $\nu_1, \dots, \nu_p$ are the multipliers for the equality constraints. The pair $(\boldsymbol{\lambda}, \boldsymbol{\nu})$ is the **dual variable**.

Definition 6.4 (Dual function). The **Lagrange dual function** is $g : \mathbb{R}^m \times \mathbb{R}^p \rightarrow [-\infty, +\infty)$,

$$g(\boldsymbol{\lambda}, \boldsymbol{\nu}) = \inf_{\mathbf{x} \in \mathbb{R}^n} L(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\nu}).$$

Its **effective domain** is $\text{dom } g = \{(\boldsymbol{\lambda}, \boldsymbol{\nu}) : g(\boldsymbol{\lambda}, \boldsymbol{\nu}) > -\infty\}$.

The infimum runs over all of $\mathbb{R}^n$, not over the feasible set $\mathcal{D}$: the dual function relaxes the original constraints and lets any violation be paid for through the multipliers.

Theorem 6.5 (Concavity of the dual function). The dual function g is concave on $\mathbb{R}^m \times \mathbb{R}^p$, regardless of whether the primal problem is convex.

Proof of Theorem 6.5

For each fixed $\mathbf{x}$, the map $(\boldsymbol{\lambda}, \boldsymbol{\nu}) \mapsto L(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\nu}) = f_0(\mathbf{x}) + \sum_i \lambda_i f_i(\mathbf{x}) + \sum_j \nu_j h_j(\mathbf{x})$ is affine in $(\boldsymbol{\lambda}, \boldsymbol{\nu})$. The dual function g is the pointwise infimum of this family of affine functions indexed by $\mathbf{x} \in \mathbb{R}^n$. Equivalently, $-g$ is the pointwise supremum of a family of affine functions, which is convex (Unit 1 §1.9.7, pointwise supremum); hence g is concave. ■

Concavity of g does not require the primal to be convex. The next subsection turns this concave function into a problem whose value lower-bounds p^* .

1.3 Weak duality

The dual function g is concave (Theorem 6.5), so maximizing it is a concave problem. The inequality multipliers $\boldsymbol{\lambda}$ are constrained to be nonnegative; the equality multipliers $\boldsymbol{\nu}$ are unrestricted.

Definition 6.6 (Dual problem). The **Lagrange dual problem** associated with the primal problem of Definition 6.1 is

$$\sup_{\boldsymbol{\lambda} \in \mathbb{R}^m, \boldsymbol{\nu} \in \mathbb{R}^p} g(\boldsymbol{\lambda}, \boldsymbol{\nu}) \quad \text{subject to } \boldsymbol{\lambda} \geq \mathbf{0}.$$

Its **optimal value** is $d^* = \sup\{g(\boldsymbol{\lambda}, \boldsymbol{\nu}) : \boldsymbol{\lambda} \geq \mathbf{0}\}$, with $d^* = -\infty$ when the feasible set $\{\boldsymbol{\lambda} \geq \mathbf{0}\} \cap \text{dom } g$ is empty and $d^* = +\infty$ when g is unbounded above on this set. A pair $(\boldsymbol{\lambda}^*, \boldsymbol{\nu}^*)$ achieving the supremum is **dual optimal**.

Theorem 6.7 (Weak duality). For every primal problem in the form of Definition 6.1, convex or not,

$$d^* \leq p^*.$$

Proof of Theorem 6.7

Let $\mathbf{x} \in \mathcal{D}$ be primal feasible, so $f_i(\mathbf{x}) \leq 0$ for all i and $h_j(\mathbf{x}) = 0$ for all j . For any $\boldsymbol{\lambda} \geq \mathbf{0}$ and $\boldsymbol{\nu} \in \mathbb{R}^p$,

$$\sum_i \lambda_i f_i(\mathbf{x}) \leq 0, \quad \sum_j \nu_j h_j(\mathbf{x}) = 0,$$

the first because each $\lambda_i \geq 0$ multiplies a nonpositive number, the second because each $h_j(\mathbf{x}) = 0$. Therefore

$$L(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\nu}) = f_0(\mathbf{x}) + \sum_i \lambda_i f_i(\mathbf{x}) + \sum_j \nu_j h_j(\mathbf{x}) \leq f_0(\mathbf{x}).$$

The dual function value is the infimum of the Lagrangian over all $\mathbf{z} \in \mathbb{R}^n$, hence is at most its value at the specific point $\mathbf{x}$:

$$g(\boldsymbol{\lambda}, \boldsymbol{\nu}) = \inf_{\mathbf{z} \in \mathbb{R}^n} L(\mathbf{z}, \boldsymbol{\lambda}, \boldsymbol{\nu}) \leq L(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\nu}) \leq f_0(\mathbf{x}).$$

This holds for every $\mathbf{x} \in \mathcal{D}$, so taking the infimum over $\mathcal{D}$ on the right,

$$g(\boldsymbol{\lambda}, \boldsymbol{\nu}) \leq \inf_{\mathbf{x} \in \mathcal{D}} f_0(\mathbf{x}) = p^*.$$

The bound holds for every $(\boldsymbol{\lambda}, \boldsymbol{\nu})$ with $\boldsymbol{\lambda} \geq \mathbf{0}$; taking the supremum over those pairs gives $d^* = \sup_{\boldsymbol{\lambda} \geq \mathbf{0}, \boldsymbol{\nu}} g(\boldsymbol{\lambda}, \boldsymbol{\nu}) \leq p^*$. ■

Definition 6.8 (Duality gap). The **duality gap** is the nonnegative number $p^* - d^*$. The gap is **closed** (and **strong duality** is said to hold) when $p^* = d^*$; otherwise the gap is open, and d^* is only a lower bound on p^* .

Weak duality is unconditional: it holds whether or not the primal is convex, whether or not Slater's condition is satisfied, and whether or not the primal or dual optima are attained. For a difficult primal, the dual still produces a usable lower bound on p^* . The next subsection identifies the conditions under which the bound is tight.

1.4 Strong duality under Slater's condition

Slater's condition combined with primal convexity closes the duality gap. The proof needs one more piece of separation theory than Unit 1 supplied: a lemma on disjoint convex sets, derived from Unit 1's projection and supporting-hyperplane theorems.

Lemma 6.9 (Hyperplane separation of disjoint convex sets). Let $A, B \subseteq \mathbb{R}^N$ be non-empty disjoint convex sets. There exists a non-zero vector $\mathbf{a} \in \mathbb{R}^N$ such that

$$\sup_{\mathbf{y} \in B} \mathbf{a}^\top \mathbf{y} \leq \inf_{\mathbf{x} \in A} \mathbf{a}^\top \mathbf{x}.$$

Proof of Lemma 6.9

Let $C = \{\mathbf{x} - \mathbf{y} : \mathbf{x} \in A, \mathbf{y} \in B\}$. The set C is convex: if $\mathbf{x}_k - \mathbf{y}_k \in C$ for $k = 1, 2$, then for $\theta \in [0, 1]$,

$$\theta(\mathbf{x}_1 - \mathbf{y}_1) + (1 - \theta)(\mathbf{x}_2 - \mathbf{y}_2) = [\theta\mathbf{x}_1 + (1 - \theta)\mathbf{x}_2] - [\theta\mathbf{y}_1 + (1 - \theta)\mathbf{y}_2] \in C$$

by convexity of A and B . Disjointness $A \cap B = \emptyset$ gives $\mathbf{0} \notin C$. Two cases on the position of $\mathbf{0}$ relative to $\overline{C}$, the closure of C .

Case 1: $\mathbf{0} \notin \overline{C}$. The closed convex set $\overline{C}$ admits a unique closest point $\mathbf{z}^*$ to $\mathbf{0}$ with $\|\mathbf{z}^*\| > 0$ (Unit 1 §1.9.8, projection step). The variational inequality $(\mathbf{0} - \mathbf{z}^*)^\top(\mathbf{z} - \mathbf{z}^*) \leq 0$ for $\mathbf{z} \in \overline{C}$ rearranges to $(\mathbf{z}^*)^\top \mathbf{z} \geq \|\mathbf{z}^*\|^2 > 0$. Setting $\mathbf{a} = \mathbf{z}^*$ gives $\mathbf{a}^\top(\mathbf{x} - \mathbf{y}) \geq \|\mathbf{a}\|^2 > 0$ for every $\mathbf{x} \in A$,

$\mathbf{y} \in B$.

Case 2: $\mathbf{0} \in \overline{C}$. Since $\mathbf{0} \notin C$, the origin is a boundary point of $\overline{C}$. By the supporting hyperplane theorem (Unit 1 §1.9.8), there exists $\mathbf{b} \neq \mathbf{0}$ with $\mathbf{b}^\top \mathbf{z} \leq \mathbf{b}^\top \mathbf{0} = 0$ for every $\mathbf{z} \in \overline{C}$. Setting $\mathbf{a} = -\mathbf{b}$, $\mathbf{a}^\top (\mathbf{x} - \mathbf{y}) \geq 0$ for every $\mathbf{x} \in A$, $\mathbf{y} \in B$.

In both cases, $\mathbf{a}^\top \mathbf{x} \geq \mathbf{a}^\top \mathbf{y}$ for $\mathbf{x} \in A$, $\mathbf{y} \in B$, which gives the stated supremum-infimum inequality.



Theorem 6.10 (Strong duality under Slater). Suppose the primal problem of Definition 6.1 has only inequality constraints (no equality constraints, $p = 0$), is convex (each f_i convex on $\mathbb{R}^n$), has finite optimal value $p^* \in \mathbb{R}$, and satisfies Slater's condition. Then $d^* = p^*$, and the dual optimum is attained: there exists $\boldsymbol{\lambda}^* \geq \mathbf{0}$ with $g(\boldsymbol{\lambda}^*) = d^* = p^*$.

Proof of Theorem 6.10

Step 1: Value set A is convex. Define

$$A = \{(\mathbf{u}, t) \in \mathbb{R}^m \times \mathbb{R} : \exists \mathbf{x} \in \mathbb{R}^n \text{ with } f_i(\mathbf{x}) \leq u_i \text{ for } i = 1, \dots, m, f_0(\mathbf{x}) \leq t\}.$$

Take $(\mathbf{u}, t), (\mathbf{u}', t') \in A$ with witnesses $\mathbf{x}, \mathbf{x}'$. For $\theta \in [0, 1]$, set $\mathbf{z} = \theta\mathbf{x} + (1 - \theta)\mathbf{x}'$. Convexity of f_i and f_0 gives

$$f_i(\mathbf{z}) \leq \theta f_i(\mathbf{x}) + (1 - \theta)f_i(\mathbf{x}') \leq \theta u_i + (1 - \theta)u'_i, \quad f_0(\mathbf{z}) \leq \theta t + (1 - \theta)t',$$

so $\theta(\mathbf{u}, t) + (1 - \theta)(\mathbf{u}', t') \in A$ with witness $\mathbf{z}$.

Step 2: $p^\star = \inf\{t : (\mathbf{0}, t) \in A\}$. If $(\mathbf{0}, t) \in A$, the witness $\mathbf{x}$ satisfies $f_i(\mathbf{x}) \leq 0$ (so $\mathbf{x}$ is primal feasible) and $f_0(\mathbf{x}) \leq t$, hence $p^\star \leq t$. Conversely, for any primal feasible $\mathbf{x}$, $(\mathbf{0}, f_0(\mathbf{x})) \in A$, so $\inf\{t : (\mathbf{0}, t) \in A\} \leq p^\star$.

Step 3: Separate A from a half-line. Let $B = \{(\mathbf{0}, s) : s < p^\star\} \subseteq \mathbb{R}^m \times \mathbb{R}$, convex. Step 2 gives $A \cap B = \emptyset$. Lemma 6.9 produces $(\boldsymbol{\mu}, \mu_0) \neq (\mathbf{0}, 0)$ with

$$\boldsymbol{\mu}^\top \mathbf{u} + \mu_0 t \geq \mu_0 s \quad \text{for every } (\mathbf{u}, t) \in A, \ s < p^\star.$$

Fix $(\mathbf{u}, t) \in A$. As $s \rightarrow -\infty$ on the right, the LHS is constant; if $\mu_0 < 0$ then $\mu_0 s \rightarrow +\infty$ and the inequality fails. So $\mu_0 \geq 0$. With $\mu_0 \geq 0$, $\sup_{s < p^\star} \mu_0 s = \mu_0 p^\star$, giving

$$\boldsymbol{\mu}^\top \mathbf{u} + \mu_0 t \geq \mu_0 p^\star \quad \text{for every } (\mathbf{u}, t) \in A. \quad (\star)$$

Step 4: $\boldsymbol{\mu} \geq \mathbf{0}$. For any $(\mathbf{u}, t) \in A$ and $\delta > 0$, the point $(\mathbf{u} + \delta \mathbf{e}_i, t)$ is also in A (the same witness $\mathbf{x}$ has $f_i(\mathbf{x}) \leq u_i \leq u_i + \delta$). Apply $(\star)$ at the perturbed point:

$$\boldsymbol{\mu}^\top \mathbf{u} + \delta \mu_i + \mu_0 t \geq \mu_0 p^\star.$$

For this to hold for every $\delta > 0$ with the LHS otherwise fixed, $\mu_i \geq 0$ (else $\delta \rightarrow \infty$ drives the LHS to $-\infty$). Repeat for each i .

Step 5: $\mu_0 > 0$. Suppose $\mu_0 = 0$. Slater's condition supplies $\tilde{\mathbf{x}}$ with $f_i(\tilde{\mathbf{x}}) < 0$ for $i = 1, \dots, m$, and the point $(f_1(\tilde{\mathbf{x}}), \dots, f_m(\tilde{\mathbf{x}}), f_0(\tilde{\mathbf{x}})) \in A$ (witness $\tilde{\mathbf{x}}$). Apply $(\star)$ with $\mu_0 = 0$:

$$\sum_{i=1}^m \mu_i f_i(\tilde{\mathbf{x}}) \geq 0.$$

Each summand $\mu_i f_i(\tilde{\mathbf{x}}) \leq 0$ (Step 4 gives $\mu_i \geq 0$, Slater gives $f_i(\tilde{\mathbf{x}}) < 0$), so the sum is ≤ 0 . Combined: the sum equals 0, hence each summand is 0, hence $\mu_i = 0$ (since $f_i(\tilde{\mathbf{x}}) \neq 0$). Thus $(\boldsymbol{\mu}, \mu_0) = (\mathbf{0}, 0)$, contradicting Lemma 6.9. So $\mu_0 > 0$.

Step 6: Identify the multiplier. Set $\boldsymbol{\lambda}^* = \boldsymbol{\mu}/\mu_0 \in \mathbb{R}_{\geq 0}^m$. For every $\mathbf{x} \in \mathbb{R}^n$, the point $(f_1(\mathbf{x}), \dots, f_m(\mathbf{x}), f_0(\mathbf{x})) \in A$ (witness $\mathbf{x}$). Substituting into $(\star)$ and dividing by $\mu_0 > 0$:

$$\sum_{i=1}^m \lambda_i^* f_i(\mathbf{x}) + f_0(\mathbf{x}) \geq p^*, \quad \text{i.e., } L(\mathbf{x}, \boldsymbol{\lambda}^*) \geq p^* \quad \forall \mathbf{x} \in \mathbb{R}^n.$$

Taking the infimum over $\mathbf{x}$: $g(\boldsymbol{\lambda}^*) \geq p^*$.

Step 7: Combine with weak duality. Weak duality (Theorem 6.7) gives $d^* \leq p^*$. Step 6 gives $g(\boldsymbol{\lambda}^*) \geq p^*$, and $\boldsymbol{\lambda}^* \geq \mathbf{0}$ makes $g(\boldsymbol{\lambda}^*)$ a candidate for the dual supremum, so $g(\boldsymbol{\lambda}^*) \leq d^*$. Stacking the chain:

$$p^* \leq g(\boldsymbol{\lambda}^*) \leq d^* \leq p^*.$$

All inequalities are equalities; $g(\boldsymbol{\lambda}^*) = d^* = p^*$, so $\boldsymbol{\lambda}^*$ attains the dual optimum. ■

Affine equality constraints

The same argument extends to convex problems with affine equality constraints $\mathbf{Ax} = \mathbf{b}$, with Slater's condition strengthened to require strict feasibility for nonlinear inequality constraints only (affine constraints are required to be feasible, not strictly so). Hard-margin and soft-margin SVM use only inequality constraints, so Theorem 6.10 applies directly without invoking the equality extension.

Strong duality is the foundation of the SVM dual: applied to the SVM primal, Theorem 6.10 guarantees that solving the dual recovers the primal optimum. The corresponding multipliers satisfy a system of equalities and complementary-slackness conditions developed next.

1.5 KKT conditions

The KKT (Karush-Kuhn-Tucker) conditions are the first-order optimality conditions for constrained convex problems. Under the hypotheses of Theorem 6.10 together with differentiability, they characterize primal-dual optimal pairs.

Definition 6.11 (KKT conditions). For the convex inequality-constrained problem of Definition 6.1 (no equality constraints) with $f_0, f_1, \dots, f_m$ differentiable, the **Karush-Kuhn-Tucker (KKT) conditions** at a pair $(\mathbf{x}^*, \boldsymbol{\lambda}^*) \in \mathbb{R}^n \times \mathbb{R}^m$ are

- (i) **Stationarity:** $\nabla f_0(\mathbf{x}^*) + \sum_{i=1}^m \lambda_i^* \nabla f_i(\mathbf{x}^*) = \mathbf{0}$;
- (ii) **Primal feasibility:** $f_i(\mathbf{x}^*) \leq 0$ for $i = 1, \dots, m$;
- (iii) **Dual feasibility:** $\lambda_i^* \geq 0$ for $i = 1, \dots, m$;
- (iv) **Complementary slackness:** $\lambda_i^* f_i(\mathbf{x}^*) = 0$ for $i = 1, \dots, m$.

Each condition has a substantive reading:

(i) extends the unconstrained first-order condition $\nabla f_0(\mathbf{x}^*) = \mathbf{0}$ (Unit 1 §1.9.5). At a constrained optimum, $\mathbf{x}^*$ globally minimizes the Lagrangian $L(\cdot, \boldsymbol{\lambda}^*)$ over $\mathbb{R}^n$, and convexity plus differentiability force $\nabla_{\mathbf{x}} L(\mathbf{x}^*, \boldsymbol{\lambda}^*) = \mathbf{0}$. The proof of Theorem 6.12 below derives this from strong duality.

(ii) restates that $\mathbf{x}^*$ is feasible.

(iii) comes from weak duality (§1.3): if some $\lambda_i < 0$, then $\lambda_i f_i(\mathbf{x})$ in the Lagrangian rewards $f_i(\mathbf{x}) > 0$ rather than penalizes it, sending $\inf_{\mathbf{x}} L(\mathbf{x}, \boldsymbol{\lambda}) \rightarrow -\infty$. Only $\boldsymbol{\lambda} \geq \mathbf{0}$ keeps $g(\boldsymbol{\lambda})$ a finite lower bound on p^* .

(iv) encodes the equality $g(\boldsymbol{\lambda}^*) = f_0(\mathbf{x}^*)$ from strong duality. Expanding $g(\boldsymbol{\lambda}^*) = \inf_{\mathbf{x}} L(\mathbf{x}, \boldsymbol{\lambda}^*) \leq L(\mathbf{x}^*, \boldsymbol{\lambda}^*) = f_0(\mathbf{x}^*) + \sum_i \lambda_i^* f_i(\mathbf{x}^*)$ forces $\sum_i \lambda_i^* f_i(\mathbf{x}^*) \geq 0$; each summand is ≤ 0 by (iii), (ii), so all summands vanish, giving $\lambda_i^* f_i(\mathbf{x}^*) = 0$ termwise.

Theorem 6.12 (KKT, necessary). Assume the primal problem of Definition 6.1 is convex with f_0, f_i differentiable on $\mathbb{R}^n$ for $i = 0, 1, \dots, m$, has no equality constraints and finite optimal value p^* , and satisfies Slater's condition. If $\mathbf{x}^*$ is primal optimal and $\boldsymbol{\lambda}^*$ is dual optimal, then the KKT conditions of Definition 6.11 hold at $(\mathbf{x}^*, \boldsymbol{\lambda}^*)$.

Proof of Theorem 6.12

Primal feasibility (ii) and dual feasibility (iii) hold by hypothesis. By strong duality (Theo-

rem 6.10), $g(\boldsymbol{\lambda}^*) = d^* = p^* = f_0(\mathbf{x}^*)$. Expanding the dual function:

$$f_0(\mathbf{x}^*) = g(\boldsymbol{\lambda}^*) = \inf_{\mathbf{x} \in \mathbb{R}^n} L(\mathbf{x}, \boldsymbol{\lambda}^*) \leq L(\mathbf{x}^*, \boldsymbol{\lambda}^*) = f_0(\mathbf{x}^*) + \sum_{i=1}^m \lambda_i^* f_i(\mathbf{x}^*).$$

Subtracting $f_0(\mathbf{x}^*)$ gives $\sum_i \lambda_i^* f_i(\mathbf{x}^*) \geq 0$. Each summand is ≤ 0 by (ii) and (iii); the sum is therefore ≤ 0 as well. Both bounds together force every summand to be 0, which is complementary slackness (iv).

With $\sum_i \lambda_i^* f_i(\mathbf{x}^*) = 0$, the displayed inequality is an equality: $L(\mathbf{x}^*, \boldsymbol{\lambda}^*) = \inf_{\mathbf{x}} L(\mathbf{x}, \boldsymbol{\lambda}^*)$. So $\mathbf{x}^*$ globally minimizes $L(\cdot, \boldsymbol{\lambda}^*)$ on $\mathbb{R}^n$. The function $L(\cdot, \boldsymbol{\lambda}^*) = f_0 + \sum_i \lambda_i^* f_i$ is convex (nonnegative combination of convex functions; Unit 1 §1.9.7) and differentiable, so the first-order optimality corollary (Unit 1 §1.9.5) gives $\nabla_{\mathbf{x}} L(\mathbf{x}^*, \boldsymbol{\lambda}^*) = \mathbf{0}$, which is stationarity (i). ■

Theorem 6.13 (KKT, sufficient). Assume the primal problem of Definition 6.1 is convex with f_0, f_i differentiable on $\mathbb{R}^n$, and has no equality constraints. If the KKT conditions hold at $(\mathbf{x}^*, \boldsymbol{\lambda}^*)$, then $\mathbf{x}^*$ is primal optimal, $\boldsymbol{\lambda}^*$ is dual optimal, and $p^* = d^*$.

Proof of Theorem 6.13

By (iii), $L(\cdot, \boldsymbol{\lambda}^*) = f_0 + \sum_i \lambda_i^* f_i$ is a nonnegative combination of convex differentiable functions, hence convex and differentiable. Stationarity (i) is $\nabla_{\mathbf{x}} L(\mathbf{x}^*, \boldsymbol{\lambda}^*) = \mathbf{0}$, so by Unit 1's first-order optimality corollary (§1.9.5), $\mathbf{x}^*$ globally minimizes $L(\cdot, \boldsymbol{\lambda}^*)$:

$$g(\boldsymbol{\lambda}^*) = L(\mathbf{x}^*, \boldsymbol{\lambda}^*) = f_0(\mathbf{x}^*) + \sum_i \lambda_i^* f_i(\mathbf{x}^*) = f_0(\mathbf{x}^*),$$

the last equality by complementary slackness (iv). For any primal feasible $\mathbf{x}$,

$$f_0(\mathbf{x}) \geq f_0(\mathbf{x}) + \sum_i \lambda_i^* f_i(\mathbf{x}) = L(\mathbf{x}, \boldsymbol{\lambda}^*) \geq L(\mathbf{x}^*, \boldsymbol{\lambda}^*) = f_0(\mathbf{x}^*),$$

where the first inequality uses $\lambda_i^* \geq 0$ and $f_i(\mathbf{x}) \leq 0$ (primal feasibility of $\mathbf{x}$), and the second inequality is the global-minimum property of $\mathbf{x}^*$ for $L(\cdot, \boldsymbol{\lambda}^*)$. Hence $f_0(\mathbf{x}^*) \leq f_0(\mathbf{x})$ for every primal feasible $\mathbf{x}$, so $\mathbf{x}^*$ is primal optimal and $f_0(\mathbf{x}^*) = p^*$. Weak duality gives $g(\boldsymbol{\lambda}^*) \leq d^* \leq p^*$; combined with $g(\boldsymbol{\lambda}^*) = f_0(\mathbf{x}^*) = p^*$, all three are equal, so $\boldsymbol{\lambda}^*$ is dual optimal and the duality gap closes. ■

Non-differentiable case

When f_0 or some f_i is convex but not differentiable, stationarity (i) is replaced by the subgradient inclusion $\mathbf{0} \in \partial_{\mathbf{x}} L(\mathbf{x}^*, \boldsymbol{\lambda}^*)$, where $\partial_{\mathbf{x}}$ is the subdifferential in $\mathbf{x}$ (Unit 1 §1.9.5). Conditions (ii)–(iv) are unchanged. Unit 2 §2.3.2 used the subgradient form for Lasso: the unconstrained Lasso problem has a subgradient stationarity condition that yields the soft-thresholding update.

The hard-margin and soft-margin SVM problems both satisfy the hypotheses of Theorems 6.12 and 6.13: convex objective, differentiable, only inequality constraints, and Slater's condition holds (under separability for hard margin, or for any $C > 0$ in soft margin). The next sections derive the SVM dual and read the support-vector geometry directly off the KKT conditions.

2 Maximum-Margin Classifier

The hard-margin support vector machine fits a separating hyperplane to linearly separable data by maximizing the geometric distance from the closest training point.

2.1 Functional and geometric margins

Throughout this section, the training data are $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ with $\mathbf{x}_i \in \mathbb{R}^p$ and $y_i \in \{-1, +1\}$. A **linear classifier** $(\mathbf{w}, b) \in \mathbb{R}^p \times \mathbb{R}$ predicts $\text{sign}(\mathbf{w}^\top \mathbf{x} + b)$. Its **decision boundary** is the hyperplane

$$H(\mathbf{w}, b) = \{\mathbf{x} \in \mathbb{R}^p : \mathbf{w}^\top \mathbf{x} + b = 0\}.$$

Definition 6.14 (Linear separability). The training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ are **linearly separable** if there exist $\mathbf{w} \in \mathbb{R}^p$ and $b \in \mathbb{R}$ with

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) > 0, \quad i = 1, \dots, n.$$

Linear separability admits infinitely many separating hyperplanes; the SVM selects one by a margin criterion.

Definition 6.15 (Functional and geometric margins). For a linear classifier $(\mathbf{w}, b)$ with $\mathbf{w} \neq \mathbf{0}$ and a labeled point $(\mathbf{x}_i, y_i)$, the **functional margin** is

$$\gamma_i(\mathbf{w}, b) = y_i(\mathbf{w}^\top \mathbf{x}_i + b),$$

and the **geometric margin** is

$$\hat{\gamma}_i(\mathbf{w}, b) = \frac{y_i(\mathbf{w}^\top \mathbf{x}_i + b)}{\|\mathbf{w}\|}.$$

The **geometric margin of the classifier on the dataset** is $\hat{\gamma}(\mathbf{w}, b) = \min_{i=1, \dots, n} \hat{\gamma}_i(\mathbf{w}, b)$.

The geometric margin is the perpendicular distance from a point to the hyperplane $H(\mathbf{w}, b)$, derived as follows. The unit normal to $H(\mathbf{w}, b)$ is $\mathbf{w}/\|\mathbf{w}\|$ (the gradient of $\mathbf{z} \mapsto \mathbf{w}^\top \mathbf{z} + b$, normalized). For $\mathbf{x} \in \mathbb{R}^p$, write the perpendicular foot on $H(\mathbf{w}, b)$ as $\mathbf{x} - t \mathbf{w}/\|\mathbf{w}\|$ and impose membership:

$$\mathbf{w}^\top \left(\mathbf{x} - t \frac{\mathbf{w}}{\|\mathbf{w}\|} \right) + b = 0 \quad \implies \quad t = \frac{\mathbf{w}^\top \mathbf{x} + b}{\|\mathbf{w}\|}.$$

The scalar t is the signed perpendicular distance from $\mathbf{x}$ to $H(\mathbf{w}, b)$, positive on the side $\{\mathbf{w}^\top \mathbf{z} + b > 0\}$. Multiplying by $y_i \in \{-1, +1\}$ inserts a sign that is positive when the classifier's prediction $\text{sign}(\mathbf{w}^\top \mathbf{x}_i + b)$ agrees with y_i , recovering $\hat{\gamma}_i = y_i(\mathbf{w}^\top \mathbf{x}_i + b)/\|\mathbf{w}\|$. The classifier separates the training data iff $\hat{\gamma}(\mathbf{w}, b) > 0$.

Scale invariance. Replacing $(\mathbf{w}, b)$ by $(c\mathbf{w}, cb)$ for $c > 0$ multiplies the functional margin by c and the norm $\|\mathbf{w}\|$ by c , so the geometric margin $\hat{\gamma}_i$ is unchanged. The hyperplane $H(\mathbf{w}, b)$ is also unchanged. Geometric margin therefore measures a property of the decision boundary itself, independent of the parameter scaling used to represent it.

Maximum-margin objective. The hard-margin SVM searches for the classifier with the largest geometric margin:

$$\max_{\mathbf{w} \in \mathbb{R}^p, b \in \mathbb{R}} \hat{\gamma}(\mathbf{w}, b).$$

Use scale invariance to fix the normalization $\min_i y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$. Any classifier with positive geometric margin admits a unique positive rescaling that produces this equality; the rescaling does not change $\hat{\gamma}$. Under this normalization $\hat{\gamma} = 1/\|\mathbf{w}\|$, so maximizing $\hat{\gamma}$ becomes minimizing $\|\mathbf{w}\|$, equivalently minimizing $\frac{1}{2}\|\mathbf{w}\|^2$. Replacing the equality $\min_i y_i(\cdot) = 1$ by the inequalities $y_i(\cdot) \geq 1$ is without loss: at any optimum at least one constraint is tight, since otherwise a small uniform rescaling $(\mathbf{w}, b) \mapsto (c\mathbf{w}, cb)$ with $c \in (0, 1)$ would decrease $\|\mathbf{w}\|$ without violating feasibility.

Definition 6.16 (Hard-margin SVM primal). Given linearly separable training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, the **hard-margin SVM primal problem** is

$$\min_{\mathbf{w} \in \mathbb{R}^p, b \in \mathbb{R}} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{subject to } y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n.$$

The objective $\frac{1}{2} \|\mathbf{w}\|^2$ is a convex quadratic in $\mathbf{w}$ (and constant in b). Each constraint $1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b) \leq 0$ is affine in $(\mathbf{w}, b)$. The hard-margin SVM is therefore a convex quadratic program with affine inequality constraints, fitting the standard form of Definition 6.1 with $f_0(\mathbf{w}, b) = \frac{1}{2} \|\mathbf{w}\|^2$ and $f_i(\mathbf{w}, b) = 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)$.

2.2 Existence and uniqueness of the optimal hyperplane

Theorem 6.17 (Hard-margin solution). On linearly separable training data containing at least one point of each class, the hard-margin SVM primal of Definition 6.16 attains a finite optimal value $p^* > 0$, the optimizer $\mathbf{w}^*$ is unique, and the set of optimal intercepts b^* is a closed bounded interval.

Proof of Theorem 6.17

The proof uses three standard analysis facts in $\mathbb{R}^d$: every bounded sequence has a convergent subsequence (Bolzano–Weierstrass); a closed set contains the limits of its convergent sequences; a continuous function preserves limits.

Feasibility. Linear separability supplies $(\mathbf{w}_0, b_0)$ with $y_i(\mathbf{w}_0^\top \mathbf{x}_i + b_0) > 0$ for all i . Setting $c = 1/\min_i y_i(\mathbf{w}_0^\top \mathbf{x}_i + b_0) > 0$ and rescaling to $(c\mathbf{w}_0, cb_0)$ produces a point feasible for Definition 6.16.

Existence. Take a feasible minimizing sequence $\{(\mathbf{w}_k, b_k)\}$, $\frac{1}{2}\|\mathbf{w}_k\|^2 \rightarrow p^*$. The norms $\|\mathbf{w}_k\|$ are bounded above. Pick any positive-class point $(\mathbf{x}_+, +1)$ and any negative-class point $(\mathbf{x}_-, -1)$; feasibility gives

$$b_k \geq 1 - \mathbf{w}_k^\top \mathbf{x}_+, \quad b_k \leq -1 - \mathbf{w}_k^\top \mathbf{x}_-,$$

which bounds $\{b_k\}$ above and below. By compactness a subsequence converges to some $(\mathbf{w}^*, b^*)$. The feasible set is closed (a finite intersection of closed half-spaces), so $(\mathbf{w}^*, b^*)$ is feasible; continuity of $\frac{1}{2}\|\cdot\|^2$ gives $\frac{1}{2}\|\mathbf{w}^*\|^2 = p^*$. The optimal value is positive: $\mathbf{w} = \mathbf{0}$ would require $y_i b \geq 1$ for every i , impossible with both labels present.

Uniqueness of $\mathbf{w}^*$. Suppose $(\mathbf{w}_1, b_1)$ and $(\mathbf{w}_2, b_2)$ both attain p^* . The midpoint $(\frac{1}{2}(\mathbf{w}_1 +$

$\mathbf{w}_2), \frac{1}{2}(b_1 + b_2))$ is feasible since the constraints are convex. A direct expansion gives the strict-convexity identity for the squared norm:

$$\left\| \frac{1}{2}(\mathbf{w}_1 + \mathbf{w}_2) \right\|^2 = \frac{1}{2}\|\mathbf{w}_1\|^2 + \frac{1}{2}\|\mathbf{w}_2\|^2 - \frac{1}{4}\|\mathbf{w}_1 - \mathbf{w}_2\|^2 = 2p^* - \frac{1}{4}\|\mathbf{w}_1 - \mathbf{w}_2\|^2.$$

The midpoint's objective is therefore $p^* - \frac{1}{8}\|\mathbf{w}_1 - \mathbf{w}_2\|^2$. Optimality forces this to be $\geq p^*$, so $\|\mathbf{w}_1 - \mathbf{w}_2\|^2 \leq 0$, hence $\mathbf{w}_1 = \mathbf{w}_2$.

Set of optimal b^* . The objective is constant in b , so every b with $(\mathbf{w}^*, b)$ feasible is itself optimal. That set is the intersection of the half-lines $b \geq 1 - (\mathbf{w}^*)^\top \mathbf{x}_i$ over i with $y_i = +1$ and $b \leq -1 - (\mathbf{w}^*)^\top \mathbf{x}_i$ over i with $y_i = -1$, a closed interval, bounded by the existence step's argument. ■

The optimal $\mathbf{w}^*$ is the unique normal direction of the maximum-margin hyperplane, and the hyperplane itself is uniquely positioned whenever both classes contribute at least one tight (support-vector) constraint. The parallelogram-law uniqueness step echoes the projection lemma in Unit 1 §1.9.8: both rest on strict convexity of the squared Euclidean norm.

3 The Dual SVM and Support Vectors

The hard-margin primal of Definition 6.16 has a dual quadratic program in n variables (one multiplier per training point), and the KKT conditions identify a small subset of training points as the support vectors that determine the decision boundary.

3.1 Lagrangian and dual derivation

Apply Definition 6.3 to the hard-margin primal. The constraints $f_i(\mathbf{w}, b) = 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b) \leq 0$ carry multipliers $\lambda_i \geq 0$, and the Lagrangian is

$$L(\mathbf{w}, b, \boldsymbol{\lambda}) = \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^n \lambda_i [1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)] = \frac{1}{2} \|\mathbf{w}\|^2 - \mathbf{w}^\top \sum_{i=1}^n \lambda_i y_i \mathbf{x}_i - b \sum_{i=1}^n \lambda_i y_i + \sum_{i=1}^n \lambda_i.$$

The hard-margin primal is convex with affine inequality constraints; Slater's condition reduces to feasibility (Definition 6.2), ensured by linear separability. By Theorem 6.10, strong duality holds and a dual optimum is attained. By Theorem 6.12, condition (i) of Definition 6.11 (stationarity) holds at

the primal-dual optimum:

$$\nabla_{\mathbf{w}} L = \mathbf{w}^* - \sum_{i=1}^n \lambda_i^* y_i \mathbf{x}_i = \mathbf{0}, \quad \frac{\partial L}{\partial b} = - \sum_{i=1}^n \lambda_i^* y_i = 0.$$

The first equation expresses $\mathbf{w}^*$ as a linear combination of the training inputs; the second is an equality constraint on the dual variable.

Theorem 6.18 (Hard-margin SVM dual). The Lagrange dual of the hard-margin SVM primal of Definition 6.16 is

$$\max_{\boldsymbol{\lambda} \in \mathbb{R}^n} \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j \quad \text{subject to} \quad \lambda_i \geq 0 \quad (i = 1, \dots, n), \quad \sum_{i=1}^n \lambda_i y_i = 0.$$

The dual optimal value equals the primal optimal value, $d^* = p^*$, and from any dual-optimal $\boldsymbol{\lambda}^*$ the primal weight vector is recovered by $\mathbf{w}^* = \sum_{i=1}^n \lambda_i^* y_i \mathbf{x}_i$.

Proof of Theorem 6.18

The Lagrangian is convex in $(\mathbf{w}, b)$ for every fixed $\boldsymbol{\lambda} \geq \mathbf{0}$ (a positive-semi-definite quadratic in $\mathbf{w}$ plus an affine term). The dual function is

$$g(\boldsymbol{\lambda}) = \inf_{\mathbf{w}, b} L(\mathbf{w}, b, \boldsymbol{\lambda}).$$

Write $\tilde{\mathbf{x}}(\boldsymbol{\lambda}) = \sum_i \lambda_i y_i \mathbf{x}_i$ and $c(\boldsymbol{\lambda}) = \sum_i \lambda_i y_i$. The Lagrangian rewrites as

$$L = \frac{1}{2} \|\mathbf{w}\|^2 - \mathbf{w}^\top \tilde{\mathbf{x}}(\boldsymbol{\lambda}) - b c(\boldsymbol{\lambda}) + \sum_i \lambda_i.$$

The term $-b c(\boldsymbol{\lambda})$ is linear in b . If $c(\boldsymbol{\lambda}) \neq 0$, sending $b \rightarrow \pm\infty$ in the appropriate sign drives $L \rightarrow -\infty$, so $g(\boldsymbol{\lambda}) = -\infty$ and $\boldsymbol{\lambda} \notin \text{dom } g$.

Restrict to $\boldsymbol{\lambda}$ with $c(\boldsymbol{\lambda}) = 0$. The Lagrangian is then constant in b and a strictly convex quadratic

in $\mathbf{w}$; its unique minimizer over $\mathbf{w}$ is $\mathbf{w}(\boldsymbol{\lambda}) = \tilde{\mathbf{x}}(\boldsymbol{\lambda})$ (set $\nabla_{\mathbf{w}}L = \mathbf{0}$). Substituting back,

$$\begin{aligned} g(\boldsymbol{\lambda}) &= \frac{1}{2}\|\tilde{\mathbf{x}}(\boldsymbol{\lambda})\|^2 - \tilde{\mathbf{x}}(\boldsymbol{\lambda})^\top \tilde{\mathbf{x}}(\boldsymbol{\lambda}) + \sum_i \lambda_i \\ &= -\frac{1}{2}\|\tilde{\mathbf{x}}(\boldsymbol{\lambda})\|^2 + \sum_i \lambda_i \\ &= \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j. \end{aligned}$$

The dual problem $\max_{\boldsymbol{\lambda} \geq 0} g(\boldsymbol{\lambda})$, together with the implicit domain restriction $c(\boldsymbol{\lambda}) = 0$, is exactly the displayed quadratic program. Theorem 6.10 gives $d^* = p^*$, and KKT condition (i) at the primal-dual optimum, $\mathbf{w}^* = \sum_i \lambda_i^* y_i \mathbf{x}_i$, recovers the primal weight. ■

Two features of the dual deserve notice. The objective depends on training inputs only through the inner products $\mathbf{x}_i^\top \mathbf{x}_j$, the slot kernel methods will fill. The constraint set is the nonnegative orthant in $\mathbb{R}^n$ intersected with one affine hyperplane, so the dual is an $n \times n$ convex QP regardless of input

dimension p .

3.2 KKT conditions and support vectors

The remaining KKT conditions identify which training points actually shape the decision boundary.

Theorem 6.19 (KKT for the hard-margin SVM). A triple $(\mathbf{w}^*, b^*, \boldsymbol{\lambda}^*)$ is primal-dual optimal for the hard-margin SVM of Definition 6.16 if and only if it satisfies all four KKT conditions of Definition 6.11, which in this problem read

- (i) **Stationarity:** $\mathbf{w}^* = \sum_{i=1}^n \lambda_i^* y_i \mathbf{x}_i$ and $\sum_{i=1}^n \lambda_i^* y_i = 0$;
- (ii) **Primal feasibility:** $y_i(\mathbf{w}^{*\top} \mathbf{x}_i + b^*) \geq 1$ for $i = 1, \dots, n$;
- (iii) **Dual feasibility:** $\lambda_i^* \geq 0$ for $i = 1, \dots, n$;
- (iv) **Complementary slackness:** $\lambda_i^* [1 - y_i(\mathbf{w}^{*\top} \mathbf{x}_i + b^*)] = 0$ for $i = 1, \dots, n$.

The hard-margin problem is convex, differentiable, has only affine inequality constraints, and Slater's condition reduces to feasibility (linear separability). Theorems 6.12 and 6.13 then apply directly to give the if-and-only-if statement.

Definition 6.20 (Support vectors). A training point $(\mathbf{x}_i, y_i)$ is a **support vector** if its multiplier at the dual optimum is strictly positive, $\lambda_i^* > 0$. The set of indices $\mathcal{S} = \{i : \lambda_i^* > 0\}$ is the **support set**.

Complementary slackness (iv) splits the training points into two groups:

- if $\lambda_i^* > 0$ then $y_i(\mathbf{w}^{*\top} \mathbf{x}_i + b^*) = 1$: the support vector $\mathbf{x}_i$ lies exactly on the margin boundary;
- if $y_i(\mathbf{w}^{*\top} \mathbf{x}_i + b^*) > 1$ then $\lambda_i^* = 0$: a strictly-correct point makes no contribution to $\mathbf{w}^*$.

Stationarity (i) reads $\mathbf{w}^* = \sum_i \lambda_i^* y_i \mathbf{x}_i$, which collapses to a sum over support vectors only:

$$\mathbf{w}^* = \sum_{i \in \mathcal{S}} \lambda_i^* y_i \mathbf{x}_i.$$

The decision boundary depends on the training data only through the support set; deleting any non-support point from the training set leaves $\mathbf{w}^*$ and b^* unchanged.

Recovering b^* . For any support-vector index $i \in \mathcal{S}$, complementary slackness gives $y_i(\mathbf{w}^{\star\top} \mathbf{x}_i + b^*) = 1$, hence

$$b^* = y_i - \mathbf{w}^{\star\top} \mathbf{x}_i = y_i - \sum_{j \in \mathcal{S}} \lambda_j^* y_j \mathbf{x}_j^\top \mathbf{x}_i \quad (i \in \mathcal{S}),$$

using $1/y_i = y_i$ for $y_i \in \{-1, +1\}$. Numerical implementations average this expression over all $i \in \mathcal{S}$ to reduce floating-point noise from any single support vector.

3.3 Margin width and the dual decision function

The support-vector characterization of §3.2 has two geometric consequences for the maximum-margin classifier.

Margin width. Under the normalization $\min_i y_i(\mathbf{w}^{\star\top} \mathbf{x}_i + b^*) = 1$, the decision boundary $\{\mathbf{x} : \mathbf{w}^{\star\top} \mathbf{x} + b^* = 0\}$ sits between the two parallel margin hyperplanes

$$H_+ = \{\mathbf{x} : \mathbf{w}^{\star\top} \mathbf{x} + b^* = +1\}, \quad H_- = \{\mathbf{x} : \mathbf{w}^{\star\top} \mathbf{x} + b^* = -1\}.$$

The Euclidean distance between H_+ and H_- is $2/\|\mathbf{w}^*\|$, the **margin width**. Maximizing the margin width is therefore equivalent to minimizing $\|\mathbf{w}^*\|$, recovering the primal objective of Definition 6.16.

Decision function in dual form. Substituting $\mathbf{w}^* = \sum_{i \in \mathcal{S}} \lambda_i^* y_i \mathbf{x}_i$ into the linear classifier yields the prediction function

$$f^*(\mathbf{x}) = \mathbf{w}^{*\top} \mathbf{x} + b^* = \sum_{i \in \mathcal{S}} \lambda_i^* y_i \mathbf{x}_i^\top \mathbf{x} + b^*, \quad \hat{y}(\mathbf{x}) = \text{sign}(f^*(\mathbf{x})).$$

Both training (the dual QP of Theorem 6.18) and prediction depend on the input data only through inner products $\mathbf{x}_i^\top \mathbf{x}_j$ and $\mathbf{x}_i^\top \mathbf{x}$. Kernel methods exploit this.

4 Soft-Margin SVM and Hinge Loss

The hard-margin SVM requires linearly separable data. Real data rarely is; the soft-margin extension allows individual training points to violate their margin constraint by a controlled amount.

4.1 Slack variables and the soft-margin primal

For each training point $(\mathbf{x}_i, y_i)$, introduce a slack variable $\xi_i \geq 0$ measuring how far the point falls short of the margin boundary. The relaxed margin constraint is

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

The value of ξ_i records the violation size:

Point status	Slack ξ_i
Correctly classified, outside margin slab ($y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$)	0
Inside margin slab	$0 < \xi_i \leq 1$
Misclassified (wrong side of decision boundary)	$\xi_i > 1$

Definition 6.21 (Soft-margin SVM primal). For training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ with $y_i \in \{-1, +1\}$ and a regularization parameter $C > 0$, the **soft-margin SVM primal** is

$$\min_{\mathbf{w} \in \mathbb{R}^p, b \in \mathbb{R}, \boldsymbol{\xi} \in \mathbb{R}^n} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad \text{subject to} \quad \begin{cases} y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, & i = 1, \dots, n, \\ \xi_i \geq 0, & i = 1, \dots, n. \end{cases}$$

$\frac{1}{2} \|\mathbf{w}\|^2$ favours a wide margin; $C \sum_i \xi_i$ penalizes violations. Large C pushes toward few violations and recovers the hard-margin solution as $C \rightarrow \infty$ on separable data; small C tolerates more violations for a wider margin.

The problem is convex: a convex-quadratic-plus-linear objective with $2n$ affine inequality constraints and no equalities. Feasibility is automatic ($\mathbf{w} = \mathbf{0}$, $b = 0$, $\xi_i = 1$ gives objective Cn); Slater reduces

to feasibility for affine constraints (Definition 6.2), so Theorem 6.10 applies for every $C > 0$ and every dataset. The hard-margin's linear-separability requirement is replaced by universal feasibility of slacks: a non-separable dataset is handled at finite cost, with positive ξ_i^* allocated to inseparable points.

4.2 Soft-margin dual and the box constraint

The Lagrangian carries one nonnegative multiplier per constraint: $\lambda_i \geq 0$ for the margin-with-slack constraint and $\mu_i \geq 0$ for the slack-nonnegativity constraint $-\xi_i \leq 0$:

$$L(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \xi_i + \sum_i \lambda_i [1 - \xi_i - y_i(\mathbf{w}^\top \mathbf{x}_i + b)] - \sum_i \mu_i \xi_i.$$

Rearranging,

$$L = \frac{1}{2} \|\mathbf{w}\|^2 - \mathbf{w}^\top \sum_i \lambda_i y_i \mathbf{x}_i - b \sum_i \lambda_i y_i + \sum_i \xi_i (C - \lambda_i - \mu_i) + \sum_i \lambda_i.$$

Theorem 6.22 (Soft-margin SVM dual). The Lagrange dual of the soft-margin SVM primal of

Definition 6.21 is

$$\max_{\boldsymbol{\lambda} \in \mathbb{R}^n} \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j \quad \text{subject to} \quad 0 \leq \lambda_i \leq C \quad (i = 1, \dots, n), \quad \sum_{i=1}^n \lambda_i y_i = 0.$$

The dual optimal value equals the primal optimal value, $d^* = p^*$, and the primal weight vector is recovered by $\mathbf{w}^* = \sum_{i=1}^n \lambda_i^* y_i \mathbf{x}_i$.

Proof of Theorem 6.22

Both b and $\boldsymbol{\xi}$ enter L linearly; for the inner infimum over them to be finite, the corresponding coefficients must vanish:

$$\frac{\partial L}{\partial b} = - \sum_i \lambda_i y_i = 0, \quad \frac{\partial L}{\partial \xi_i} = C - \lambda_i - \mu_i = 0 \quad \text{for every } i.$$

If either condition fails, the linear term drives L to $-\infty$ along the appropriate direction in b or ξ_i , so the dual point falls outside $\text{dom } g$. The second condition gives $\mu_i = C - \lambda_i$; combined with $\mu_i \geq 0$, this produces the upper bound $\lambda_i \leq C$, and the slack-nonnegativity multiplier $\boldsymbol{\mu}$

disappears from the dual.

Setting $\nabla_{\mathbf{w}} L = \mathbf{0}$ gives $\mathbf{w}(\boldsymbol{\lambda}) = \sum_i \lambda_i y_i \mathbf{x}_i$. Substituting back, with the slack term $\xi_i(C - \lambda_i - \mu_i)$ identically zero,

$$g(\boldsymbol{\lambda}) = -\frac{1}{2} \|\mathbf{w}(\boldsymbol{\lambda})\|^2 + \sum_i \lambda_i = \sum_i \lambda_i - \frac{1}{2} \sum_{i,j} \lambda_i \lambda_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j.$$

The dual problem is $\max_{\boldsymbol{\lambda}} g(\boldsymbol{\lambda})$ over the box $0 \leq \lambda_i \leq C$ plus the equality constraint $\sum_i \lambda_i y_i = 0$. Theorem 6.10, whose hypotheses were verified at the close of §4.1, gives $d^* = p^*$ and the recovery formula for $\mathbf{w}^*$. ■

The soft-margin dual differs from the hard-margin dual (Theorem 6.18) only by adding the upper bound $\lambda_i \leq C$. As $C \rightarrow \infty$, the bound becomes inactive and the dual reduces to the hard-margin one; finite C caps each point's influence on $\mathbf{w}^*$.

Three classes of training points. Complementary slackness for the two constraints reads

$$\lambda_i^* [1 - \xi_i^* - y_i(\mathbf{w}^{*\top} \mathbf{x}_i + b^*)] = 0, \quad \mu_i^* \xi_i^* = (C - \lambda_i^*) \xi_i^* = 0.$$

The pair partitions training points by the value of λ_i^* :

- $\lambda_i^* = 0$: $\mu_i^* = C > 0$ forces $\xi_i^* = 0$, and the margin constraint holds with $y_i(\mathbf{w}^{*\top} \mathbf{x}_i + b^*) \geq 1$. The point sits outside the margin slab and contributes nothing to $\mathbf{w}^*$.
- $0 < \lambda_i^* < C$ (**margin support vector**): $\mu_i^* = C - \lambda_i^* > 0$ forces $\xi_i^* = 0$, and the first complementary slackness equation gives $y_i(\mathbf{w}^{*\top} \mathbf{x}_i + b^*) = 1$. The point sits exactly on the margin boundary.
- $\lambda_i^* = C$ (**bound support vector**): $\mu_i^* = 0$ permits $\xi_i^* \geq 0$, and $y_i(\mathbf{w}^{*\top} \mathbf{x}_i + b^*) = 1 - \xi_i^*$. The point sits inside the margin slab when $0 < \xi_i^* \leq 1$ and is misclassified when $\xi_i^* > 1$.

Recovering b^* . A margin support vector ($0 < \lambda_i^* < C$) has $\xi_i^* = 0$ and $y_i(\mathbf{w}^{*\top} \mathbf{x}_i + b^*) = 1$, hence $b^* = y_i - \mathbf{w}^{*\top} \mathbf{x}_i$. Bound support vectors do not pin b^* because their slack value is not determined directly from $\boldsymbol{\lambda}^*$; numerical implementations average the formula over the margin support vectors only.

4.3 Hinge loss reformulation

The soft-margin primal can be rewritten as a regularized empirical risk minimization in $(\mathbf{w}, b)$ alone, with the slack variables eliminated and a specific loss function exposed.

Lemma 6.23 (Optimal slack equals hinge loss). At any optimum of the soft-margin primal of Definition 6.21, the optimal slack for training point $(\mathbf{x}_i, y_i)$ is

$$\xi_i^* = [1 - y_i(\mathbf{w}^{*\top} \mathbf{x}_i + b^*)]_+ = \max(0, 1 - y_i(\mathbf{w}^{*\top} \mathbf{x}_i + b^*)).$$

Proof of Lemma 6.23

Fix $(\mathbf{w}^*, b^*)$ at the optimum. The slack ξ_i enters the primal only through the term $C\xi_i$ in the objective and the per-point constraints $\xi_i \geq 0$ and $\xi_i \geq 1 - y_i(\mathbf{w}^{*\top} \mathbf{x}_i + b^*)$. With $C > 0$, minimizing the objective in ξ_i alone (the other variables fixed) is achieved at the smallest feasible value, which is $\max(0, 1 - y_i(\mathbf{w}^{*\top} \mathbf{x}_i + b^*))$. ■

Definition 6.24 (Hinge loss). The **hinge loss** for binary classification with labels $y \in \{-1, +1\}$ and a real-valued score $f(\mathbf{x})$ is

$$\ell_{\text{hinge}}(y, f(\mathbf{x})) = [1 - y f(\mathbf{x})]_+ = \max(0, 1 - y f(\mathbf{x})).$$

The loss is zero on points classified correctly with score magnitude at least 1; for $y f(\mathbf{x}) < 1$ the loss grows linearly in the gap.

Substituting Lemma 6.23 into the soft-margin primal eliminates $\boldsymbol{\xi}$ and yields an equivalent unconstrained reformulation:

$$\min_{\mathbf{w} \in \mathbb{R}^p, b \in \mathbb{R}} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n [1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)]_+.$$

This is regularized empirical risk minimization: the per-point hinge loss summed over the training set, plus an ℓ_2 regularizer on $\mathbf{w}$. Dividing through by Cn rewrites the objective in the more familiar regularized-risk form

$$\frac{1}{n} \sum_{i=1}^n [1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)]_+ + \frac{1}{2Cn} \|\mathbf{w}\|^2,$$

in which the regularization weight is $1/(2Cn)$, decreasing in C .

Comparison with other classification losses. The hinge loss is one of three convex surrogates for the intractable 0/1 loss seen in this course:

Loss	Formula at score $f(\mathbf{x})$	Differentiable?
0/1 loss	$\mathbb{1}\{yf(\mathbf{x}) < 0\}$	no, non-convex
Logistic loss	$\log(1 + \exp(-yf(\mathbf{x})))$	yes, convex
Hinge loss	$[1 - yf(\mathbf{x})]_+$	no (kink at $yf = 1$), convex

Logistic loss (Unit 3 §3.1.3) is smooth, fitting gradient methods in the primal. Hinge loss has a kink at $yf(\mathbf{x}) = 1$: the primal is non-differentiable, but the dual (Theorem 6.22) is a smooth QP. Standard SVM solvers exploit this asymmetry by solving the dual.

5 Kernels

The dual of the soft-margin SVM (Theorem 6.22) depends on the input data only through the pairwise inner products $\mathbf{x}_i^\top \mathbf{x}_j$. Replacing those inner products with a more expressive symmetric function of $\mathbf{x}_i$

and $\mathbf{x}_j$ extends the SVM to nonlinear decision boundaries without changing the optimization problem's size.

5.1 Feature maps and the kernel trick

Definition 6.25 (Feature map). A **feature map** from input space $\mathbb{R}^p$ to feature space $\mathbb{R}^N$ is a function $\phi : \mathbb{R}^p \rightarrow \mathbb{R}^N$ with N possibly $\gg p$. The induced classifier searches for a separating hyperplane in feature space:

$$f(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x}) + b, \quad \mathbf{w} \in \mathbb{R}^N, b \in \mathbb{R}.$$

The soft-margin SVM in the feature space replaces each $\mathbf{x}_i$ by $\phi(\mathbf{x}_i)$ and each inner product $\mathbf{x}_i^\top \mathbf{x}_j$ by $\phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$. The dual of Theorem 6.22 becomes

$$\max_{\lambda} \sum_i \lambda_i - \frac{1}{2} \sum_{i,j} \lambda_i \lambda_j y_i y_j \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j) \quad \text{subject to } 0 \leq \lambda_i \leq C, \sum_i \lambda_i y_i = 0,$$

and the prediction function evaluates as $f^*(\mathbf{x}) = \sum_{i \in \mathcal{S}} \lambda_i^* y_i \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}) + b^*$.

Definition 6.26 (Kernel function). For a feature map $\phi : \mathbb{R}^p \rightarrow \mathbb{R}^N$, the associated **kernel function** is

$$K : \mathbb{R}^p \times \mathbb{R}^p \rightarrow \mathbb{R}, \quad K(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^\top \phi(\mathbf{x}').$$

The dual SVM and the prediction function depend on ϕ only through K . Whenever K can be evaluated more cheaply than via the explicit feature vectors $\phi(\mathbf{x}), \phi(\mathbf{x}')$, the SVM extends to high-dimensional feature spaces at no additional optimization cost. This computational shortcut is the **kernel trick**.

Example 6.1 (Quadratic kernel on $\mathbb{R}^2$). Take $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^2$ and define

$$\phi(\mathbf{x}) = (x_1^2, x_2^2, \sqrt{2}x_1x_2, \sqrt{2}x_1, \sqrt{2}x_2, 1)^\top \in \mathbb{R}^6.$$

Expanding $(\mathbf{x}^\top \mathbf{x}' + 1)^2$ in the variables x_1, x_2, x'_1, x'_2 and matching term-by-term against $\phi(\mathbf{x})^\top \phi(\mathbf{x}')$ confirms

$$\phi(\mathbf{x})^\top \phi(\mathbf{x}') = (\mathbf{x}^\top \mathbf{x}' + 1)^2.$$

The kernel $K(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^\top \mathbf{x}' + 1)^2$ evaluates a 6-dimensional inner product at the cost of one 2-dimensional inner product and a squaring. The cost saving generalizes: a degree- d polynomial

kernel on $\mathbb{R}^p$ corresponds to a feature map into a space of dimension $\binom{p+d}{d}$, evaluated in $O(p)$ time per pair instead of $O(\binom{p+d}{d})$.

Infinite-dimensional feature spaces

The Gaussian RBF kernel $K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$ corresponds to a feature map into an **infinite**-dimensional reproducing kernel Hilbert space (RKHS). The dual hides the infinite-dimensional construction: every quantity needed for training and prediction goes through K , and the PSD condition (next subsection) guarantees that ϕ exists.

The kernel trick decouples training cost from the feature-space dimension. The next subsection identifies which symmetric functions K arise from some feature map.

5.2 The PSD condition for kernels

Not every symmetric function $K : \mathbb{R}^p \times \mathbb{R}^p \rightarrow \mathbb{R}$ arises as an inner product in some feature space. The function must be **positive semi-definite**.

Definition 6.27 (Positive-semi-definite kernel). A symmetric function $K : \mathbb{R}^p \times \mathbb{R}^p \rightarrow \mathbb{R}$ is a **positive-semi-definite (PSD) kernel** if for every finite set $\{\mathbf{x}_1, \dots, \mathbf{x}_n\} \subset \mathbb{R}^p$ the Gram matrix

$$\mathbf{K} = [K(\mathbf{x}_i, \mathbf{x}_j)]_{i,j=1}^n \in \mathbb{R}^{n \times n}$$

is symmetric positive semi-definite (Unit 1, the PSD definition).

The PSD condition is necessary for K to express an inner product. If $K(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^\top \phi(\mathbf{x}')$, then for any vector $\mathbf{c} \in \mathbb{R}^n$,

$$\mathbf{c}^\top \mathbf{K} \mathbf{c} = \sum_{i,j} c_i c_j \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j) = \left\| \sum_i c_i \phi(\mathbf{x}_i) \right\|^2 \geq 0.$$

The converse holds on every finite training set, via spectral decomposition.

Theorem 6.28 (Finite-sample feature map). Let K be a PSD kernel and $\{\mathbf{x}_1, \dots, \mathbf{x}_n\} \subset \mathbb{R}^p$ any finite training set. The Gram matrix decomposes as $\mathbf{K} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^\top$ with $\mathbf{V} \in \mathbb{R}^{n \times n}$ orthogonal and $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n)$, $\lambda_i \geq 0$. Setting $\mathbf{\Phi} = \mathbf{V} \mathbf{\Lambda}^{1/2} \in \mathbb{R}^{n \times n}$, the rows of $\mathbf{\Phi}$ act as feature

vectors for the training set:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \Phi_i^\top \Phi_j, \quad i, j = 1, \dots, n.$$

Proof of Theorem 6.28

The Gram matrix $\mathbf{K}$ is symmetric PSD by hypothesis, so spectral decomposition gives $\mathbf{K} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^\top$ with $\mathbf{V}$ orthogonal and $\mathbf{\Lambda}$ diagonal nonnegative. Then

$$\mathbf{K} = \mathbf{V}\mathbf{\Lambda}^{1/2}\mathbf{\Lambda}^{1/2}\mathbf{V}^\top = \Phi\Phi^\top,$$

and $K_{ij} = (\Phi\Phi^\top)_{ij} = \Phi_i^\top \Phi_j$. ■

The theorem says PSD is sufficient on every finite training set: the SVM dual with PSD kernel K has the form

$$\max_{\lambda} \sum_i \lambda_i - \frac{1}{2} \boldsymbol{\lambda}^\top \mathbf{Y} \mathbf{K} \mathbf{Y} \boldsymbol{\lambda} \quad \text{subject to } 0 \leq \lambda_i \leq C, \quad \sum_i \lambda_i y_i = 0,$$

where $\mathbf{Y} = \text{diag}(y_1, \dots, y_n)$. The matrix $\mathbf{Y} \mathbf{K} \mathbf{Y}$ is PSD whenever $\mathbf{K}$ is, so the dual objective is concave

and the dual is a convex QP solvable by off-the-shelf solvers. The infinite-dimensional feature map underlying (for example) the RBF kernel is invisible to the solver, which works exclusively with the $n \times n$ Gram matrix.

Common kernels.

Kernel	Formula	Feature space
Linear	$K(\mathbf{x}, \mathbf{x}') = \mathbf{x}^\top \mathbf{x}'$	$\mathbb{R}^p$, the input space itself
Polynomial	$K(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^\top \mathbf{x}' + c)^d, c \geq 0, d \in \mathbb{N}$	finite, $\binom{p+d}{d}$ -dim
RBF	$K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \ \mathbf{x} - \mathbf{x}'\ ^2), \gamma > 0$	infinite-dimensional

The polynomial kernel admits an explicit feature map (Example 6.1 gives the $p = 2, d = 2$ case). The RBF kernel does not admit a finite-dimensional explicit feature map. To establish that the RBF Gram matrix is PSD for every finite training set, we record closure properties of PSD matrices and assemble the RBF kernel from them.

Lemma 6.29 (Closure properties of PSD matrices). Let $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times n}$ be symmetric PSD.

- (i) (Sum.) $\mathbf{A} + \mathbf{B}$ is PSD.
- (ii) (Schur product theorem.) The entrywise product $\mathbf{A} \circ \mathbf{B}$, defined by $(\mathbf{A} \circ \mathbf{B})_{ij} = A_{ij}B_{ij}$, is PSD.
- (iii) (Outer rescaling.) For any $\mathbf{g} \in \mathbb{R}^n$, the matrix with entries $g_i A_{ij} g_j$ is PSD.
- (iv) (Entrywise limits.) If $\mathbf{A}_k$ is PSD for every k and $\mathbf{A}_k \rightarrow \mathbf{A}$ entrywise, then $\mathbf{A}$ is PSD.

Proof of Lemma 6.29

- (i) $\mathbf{v}^\top (\mathbf{A} + \mathbf{B}) \mathbf{v} = \mathbf{v}^\top \mathbf{A} \mathbf{v} + \mathbf{v}^\top \mathbf{B} \mathbf{v} \geq 0$.
- (ii) Spectral decomposition gives $\mathbf{A} = \sum_r \mathbf{u}_r \mathbf{u}_r^\top$ and $\mathbf{B} = \sum_s \mathbf{w}_s \mathbf{w}_s^\top$ with the nonnegative eigenvalues absorbed into the vectors. Then

$$A_{ij} B_{ij} = \sum_{r,s} u_{r,i} u_{r,j} w_{s,i} w_{s,j} = \sum_{r,s} (u_{r,i} w_{s,i}) (u_{r,j} w_{s,j}),$$

so $\mathbf{A} \circ \mathbf{B} = \sum_{r,s} \mathbf{z}_{rs} \mathbf{z}_{rs}^\top$ with $\mathbf{z}_{rs} = \mathbf{u}_r \circ \mathbf{w}_s$, a nonnegative sum of rank-one PSD matrices.

- (iii) Let $\mathbf{A}'$ have entries $A'_{ij} = g_i A_{ij} g_j$. Then $\mathbf{v}^\top \mathbf{A}' \mathbf{v} = \sum_{ij} (v_i g_i) A_{ij} (v_j g_j) = (\mathbf{g} \circ \mathbf{v})^\top \mathbf{A} (\mathbf{g} \circ \mathbf{v}) \geq 0$.

(iv) For fixed $\mathbf{v}$, $\mathbf{v}^\top \mathbf{A}_k \mathbf{v} \geq 0$ for every k , and the sequence converges to $\mathbf{v}^\top \mathbf{A} \mathbf{v}$; the limit is therefore nonnegative. ■

Theorem 6.30 (The RBF kernel is PSD). For every $\gamma > 0$ and every finite $\{\mathbf{x}_1, \dots, \mathbf{x}_n\} \subset \mathbb{R}^p$, the Gram matrix $K_{ij} = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$ is positive semi-definite.

Proof of Theorem 6.30

Expand $\|\mathbf{x}_i - \mathbf{x}_j\|^2 = \|\mathbf{x}_i\|^2 - 2\mathbf{x}_i^\top \mathbf{x}_j + \|\mathbf{x}_j\|^2$, so

$$K_{ij} = e^{-\gamma \|\mathbf{x}_i\|^2} e^{2\gamma \mathbf{x}_i^\top \mathbf{x}_j} e^{-\gamma \|\mathbf{x}_j\|^2}.$$

Set $g_i = e^{-\gamma \|\mathbf{x}_i\|^2}$ and $M_{ij} = e^{2\gamma \mathbf{x}_i^\top \mathbf{x}_j}$, giving $K_{ij} = g_i M_{ij} g_j$. By Lemma 6.29(iii) it suffices to show $\mathbf{M}$ is PSD.

The linear-kernel matrix $L_{ij} = \mathbf{x}_i^\top \mathbf{x}_j$ is the Gram matrix of the identity feature map and is PSD. For each integer $m \geq 1$, the entrywise power $L_{ij}^{\circ m} = L_{ij}^m$ is PSD by $(m-1)$ applications of Lemma 6.29(ii); the $m=0$ case is the all-ones matrix $\mathbf{1}\mathbf{1}^\top$, also PSD. The exponential admits

the entrywise convergent expansion

$$M_{ij} = \sum_{m=0}^{\infty} \frac{(2\gamma)^m}{m!} L_{ij}^m.$$

Every partial sum is a nonnegative combination of PSD matrices, hence PSD by Lemma 6.29(i); the entrywise limit $\mathbf{M}$ is PSD by Lemma 6.29(iv). Therefore $\mathbf{K}$ is PSD. ■

The hyperparameter γ controls locality: small γ approaches a constant kernel (every pair looks similar); large γ concentrates near the diagonal (every pair looks dissimilar unless near-coincident).

5.3 Representer theorem

The representer theorem states that the optimal weight vector of any kernel-based regularized risk minimization lies in the span of the training feature vectors. The optimization therefore reduces to choosing n coefficients, independent of the feature-space dimension. The proof rests on a linear-algebra fact about orthogonal decomposition.

Lemma 6.31 (Orthogonal decomposition). For any subspace $\mathcal{V} \subseteq \mathbb{R}^N$ and any $\mathbf{w} \in \mathbb{R}^N$, there is a unique decomposition

$$\mathbf{w} = \mathbf{w}_{\parallel} + \mathbf{w}_{\perp}, \quad \mathbf{w}_{\parallel} \in \mathcal{V}, \quad \mathbf{w}_{\perp} \in \mathcal{V}^{\perp} = \{\mathbf{u} \in \mathbb{R}^N : \mathbf{u}^{\top} \mathbf{v} = 0 \, \forall \mathbf{v} \in \mathcal{V}\}.$$

It satisfies $\mathbf{w}_{\parallel}^{\top} \mathbf{w}_{\perp} = 0$ and the Pythagorean identity $\|\mathbf{w}\|^2 = \|\mathbf{w}_{\parallel}\|^2 + \|\mathbf{w}_{\perp}\|^2$.

Proof of Lemma 6.31

Let $\{\mathbf{v}_1, \dots, \mathbf{v}_k\}$ be an orthonormal basis of $\mathcal{V}$, obtained from any spanning set by Gram–Schmidt. Set

$$\mathbf{w}_{\parallel} = \sum_{j=1}^k (\mathbf{w}^{\top} \mathbf{v}_j) \mathbf{v}_j, \quad \mathbf{w}_{\perp} = \mathbf{w} - \mathbf{w}_{\parallel}.$$

Then $\mathbf{w}_{\parallel} \in \mathcal{V}$ by construction, and $\mathbf{w}_{\perp}^{\top} \mathbf{v}_j = \mathbf{w}^{\top} \mathbf{v}_j - \mathbf{w}^{\top} \mathbf{v}_j = 0$ for every j , so $\mathbf{w}_{\perp} \in \mathcal{V}^{\perp}$. For uniqueness, suppose $\mathbf{w} = \mathbf{u}_{\parallel} + \mathbf{u}_{\perp}$ is another such decomposition; then $\mathbf{w}_{\parallel} - \mathbf{u}_{\parallel} = \mathbf{u}_{\perp} - \mathbf{w}_{\perp} \in \mathcal{V} \cap \mathcal{V}^{\perp}$, and any $\mathbf{v} \in \mathcal{V} \cap \mathcal{V}^{\perp}$ satisfies $\mathbf{v}^{\top} \mathbf{v} = 0$ hence $\mathbf{v} = \mathbf{0}$. The Pythagorean identity follows

from $\|\mathbf{w}_{\parallel} + \mathbf{w}_{\perp}\|^2 = \|\mathbf{w}_{\parallel}\|^2 + 2\mathbf{w}_{\parallel}^{\top} \mathbf{w}_{\perp} + \|\mathbf{w}_{\perp}\|^2$ using $\mathbf{w}_{\parallel}^{\top} \mathbf{w}_{\perp} = 0$. ■

Theorem 6.32 (Representer theorem). Let $\phi : \mathbb{R}^p \rightarrow \mathbb{R}^N$ be a feature map (with N possibly large). Consider the regularized risk minimization

$$\min_{\mathbf{w} \in \mathbb{R}^N, b \in \mathbb{R}} \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^n L(y_i, \mathbf{w}^{\top} \phi(\mathbf{x}_i) + b),$$

where L is any loss function (no convexity, smoothness, or sign assumptions). Every optimum $\mathbf{w}^{\star}$ admits the representation

$$\mathbf{w}^{\star} = \sum_{i=1}^n \alpha_i \phi(\mathbf{x}_i)$$

for some coefficients $\alpha_1, \dots, \alpha_n \in \mathbb{R}$.

Proof of Theorem 6.32

Let $\mathcal{V} = \text{span}\{\phi(\mathbf{x}_1), \dots, \phi(\mathbf{x}_n)\} \subseteq \mathbb{R}^N$, a subspace of dimension at most n . By Lemma 6.31, every $\mathbf{w} \in \mathbb{R}^N$ has a unique decomposition $\mathbf{w} = \mathbf{w}_{\parallel} + \mathbf{w}_{\perp}$ with $\mathbf{w}_{\parallel} \in \mathcal{V}$, $\mathbf{w}_{\perp} \in \mathcal{V}^{\perp}$. The prediction at every training point uses only the parallel component:

$$\mathbf{w}^{\top} \phi(\mathbf{x}_i) = \mathbf{w}_{\parallel}^{\top} \phi(\mathbf{x}_i) + \mathbf{w}_{\perp}^{\top} \phi(\mathbf{x}_i) = \mathbf{w}_{\parallel}^{\top} \phi(\mathbf{x}_i),$$

where the second equality uses $\phi(\mathbf{x}_i) \in \mathcal{V}$ and $\mathbf{w}_{\perp} \perp \mathcal{V}$. The fitting sum $\sum_i L(y_i, \mathbf{w}^{\top} \phi(\mathbf{x}_i) + b)$ therefore depends only on $\mathbf{w}_{\parallel}$, not on $\mathbf{w}_{\perp}$.

The regularizer obeys $\|\mathbf{w}\|^2 = \|\mathbf{w}_{\parallel}\|^2 + \|\mathbf{w}_{\perp}\|^2 \geq \|\mathbf{w}_{\parallel}\|^2$, with equality if and only if $\mathbf{w}_{\perp} = \mathbf{0}$. If $\mathbf{w}^{\star}$ were optimal with $\mathbf{w}_{\perp}^{\star} \neq \mathbf{0}$, replacing it by $\mathbf{w}_{\parallel}^{\star}$ would leave the fitting term unchanged and strictly decrease the regularizer, strictly improving the objective: contradiction. Hence $\mathbf{w}_{\perp}^{\star} = \mathbf{0}$ at every optimum, and $\mathbf{w}^{\star} \in \mathcal{V}$. By definition of $\mathcal{V}$, $\mathbf{w}^{\star} = \sum_i \alpha_i \phi(\mathbf{x}_i)$ for some $\boldsymbol{\alpha} \in \mathbb{R}^n$. ■

Two consequences. First, the optimum lives in an at-most- n -dimensional subspace pinned by the training data, regardless of the feature space's nominal dimension; this is what makes the kernel trick computationally feasible. Second, the same conclusion applies to ridge regression, kernel ridge

regression, kernel logistic regression, and any other regularized risk minimization sharing the squared-norm regularizer; the loss function's identity does not enter the proof.

Reproducing kernel Hilbert spaces

The proof uses only the inner-product and orthogonal-decomposition structure, which carry over to any separable Hilbert space. Applied to the reproducing kernel Hilbert space (RKHS) attached to a PSD kernel K , the theorem becomes the version stated in [Schölkopf and Smola \(2002\)](#): every optimum of the RKHS-regularized problem has $\mathbf{w}^* = \sum_i \alpha_i K(\mathbf{x}_i, \cdot)$. The infinite-dimensional construction is hidden behind the kernel, exactly as in §5.2.

5.4 Kernel SVM in the dual

Combining §§5.1–5.3: the soft-margin SVM with feature map ϕ corresponding to a PSD kernel K has dual problem

$$\max_{\lambda \in \mathbb{R}^n} \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j) \quad \text{subject to } 0 \leq \lambda_i \leq C, \sum_{i=1}^n \lambda_i y_i = 0,$$

and prediction function

$$f^*(\mathbf{x}) = \sum_{i \in \mathcal{S}} \lambda_i^* y_i K(\mathbf{x}_i, \mathbf{x}) + b^*, \quad \hat{y}(\mathbf{x}) = \text{sign}(f^*(\mathbf{x})).$$

The intercept b^* is recovered from any margin support vector i (one with $0 < \lambda_i^* < C$):

$$b^* = y_i - \sum_{j \in \mathcal{S}} \lambda_j^* y_j K(\mathbf{x}_j, \mathbf{x}_i).$$

The kernel SVM is the linear SVM with $\mathbf{x}_i^\top \mathbf{x}_j$ replaced by $K(\mathbf{x}_i, \mathbf{x}_j)$ everywhere inner products appear (Gram matrix, prediction, b^* recovery). The feature map ϕ is never accessed; the algorithm operates only on K values and training labels.

Why this works. The representer theorem (Theorem 6.32) underwrites the substitution. The optimal feature-space weight vector

$$\mathbf{w}^* = \sum_{i \in \mathcal{S}} \lambda_i^* y_i \phi(\mathbf{x}_i)$$

lies in the span of training feature vectors, so $\mathbf{w}^{\star\top} \phi(\mathbf{x}) = \sum_{i \in \mathcal{S}} \lambda_i^* y_i \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}) = \sum_{i \in \mathcal{S}} \lambda_i^* y_i K(\mathbf{x}_i, \mathbf{x})$ depends only on K and the training data. The PSD condition (Definition 6.27) ensures the dual is a convex QP, and strong duality (Theorem 6.10, applied in the feature space) ensures the dual optimum equals the primal optimum.

Computational cost. The dual is an $n \times n$ QP, independent of N . Training is dominated by the Gram matrix in $O(n^2 \tau(K))$ time and $O(n^2)$ memory, where $\tau(K)$ is the per-pair kernel cost; prediction takes $O(|\mathcal{S}| \tau(K))$ per point. The SVM suits small to moderate n with large N . The $O(n^2)$ memory becomes the binding constraint around $n \sim 10^4$ – 10^5 , past which kernel methods scale only via approximations (random Fourier features, Nyström) or yield to deep learning.

6 Neural Networks

A neural network is a parametrized nonlinear function $f(\mathbf{x}; \boldsymbol{\theta})$ obtained by alternating affine maps with elementwise nonlinearities. Training fits $\boldsymbol{\theta}$ by stochastic gradient descent on a non-convex empirical risk; each iteration requires $\nabla_{\boldsymbol{\theta}} L$ over a parameter vector with 10^7 or more entries. **Backpropagation** is reverse-mode automatic differentiation specialized to the layered composition: it returns $\nabla_{\boldsymbol{\theta}} L$

at the cost of one extra forward pass.

6.1 The multilayer perceptron and forward propagation

Definition 6.33 (L -layer multilayer perceptron). Fix layer widths $d_0 = p, d_1, \dots, d_L$. The L -layer multilayer perceptron (MLP) is the function $f : \mathbb{R}^p \rightarrow \mathbb{R}^{d_L}$ defined by

$$\mathbf{a}^{(0)} = \mathbf{x}, \quad \mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)}), \quad l = 1, \dots, L.$$

The parameters are $\boldsymbol{\theta} = \{\mathbf{W}^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}, \mathbf{b}^{(l)} \in \mathbb{R}^{d_l}\}_{l=1}^L$, the activation $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ is applied componentwise, and the output is $f(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{a}^{(L)}$.

The vector $\mathbf{z}^{(l)} \in \mathbb{R}^{d_l}$ is the **pre-activation** and $\mathbf{a}^{(l)} \in \mathbb{R}^{d_l}$ the **activation** of layer l . Componentwise, $z_j^{(l)} = \sum_{k=1}^{d_{l-1}} W_{jk}^{(l)} a_k^{(l-1)} + b_j^{(l)}$, so $W_{jk}^{(l)}$ is the weight from neuron k in layer $l-1$ to neuron j in layer l , and $a_j^{(l)} = \sigma(z_j^{(l)})$.

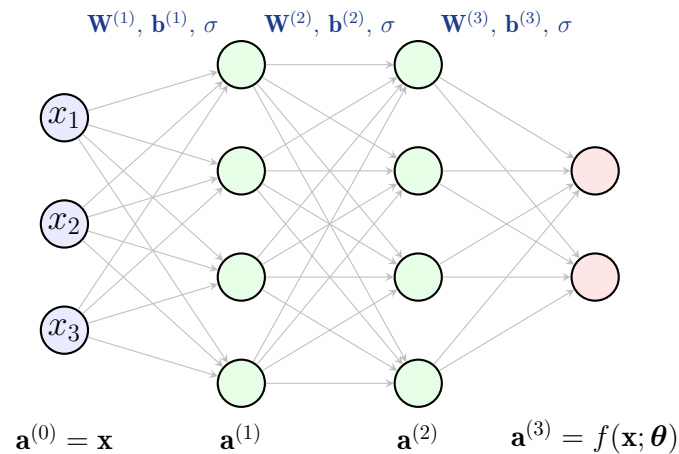


Figure 6.1: An $L = 3$ MLP with widths $d_0 = 3$, $d_1 = d_2 = 4$, $d_3 = 2$. Each blue node holds an input component, green nodes hold hidden activations, red nodes hold the output. Each arrow corresponds to one entry of the weight matrix $\mathbf{W}^{(l)}$; the bias $\mathbf{b}^{(l)}$ adds to every neuron in layer l , and σ is applied componentwise after the affine step.

Activation	$\sigma(z)$	$\sigma'(z)$
ReLU	$\max(0, z)$	$\mathbb{1}\{z > 0\}$
sigmoid	$1/(1 + e^{-z})$	$\sigma(z)(1 - \sigma(z))$
tanh	$(e^z - e^{-z})/(e^z + e^{-z})$	$1 - \tanh^2(z)$

The choice of σ at the output layer is task-dependent: identity for regression with squared loss, sigmoid for binary classification with cross-entropy, softmax (no longer componentwise) for multi-class classification with cross-entropy.

Hadamard product. For $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$, the **Hadamard product** is the componentwise vector $(\mathbf{u} \odot \mathbf{v})_j = u_j v_j$. Two identities will be used repeatedly: $\sigma(\mathbf{z})$ is itself a Hadamard product against the all-ones vector after applying σ pointwise, and for any vector $\mathbf{u}$,

$$\text{diag}(\sigma'(\mathbf{z})) \mathbf{u} = \sigma'(\mathbf{z}) \odot \mathbf{u},$$

i.e., multiplication by a diagonal matrix coincides with the Hadamard product against the diagonal entries.

Theorem 6.34 (Universal approximation; Cybenko 1989, Hornik 1991). For any compact $K \subset \mathbb{R}^p$, any continuous $f : K \rightarrow \mathbb{R}$, and any $\epsilon > 0$, there exist a width $d_1 \in \mathbb{N}$ and parameters $\boldsymbol{\theta}$ for a 2-layer MLP ($L = 2$) with any non-polynomial bounded continuous activation σ such that $\sup_{\mathbf{x} \in K} |\hat{f}(\mathbf{x}; \boldsymbol{\theta}) - f(\mathbf{x})| < \epsilon$.

The theorem says any continuous function on a compact set is approximable by an MLP, but it does not bound the required width or supply the parameters. Practical neural networks therefore depend on (a) optimization of $\boldsymbol{\theta}$ over training data, and (b) efficient gradient computation, the subjects of the remaining subsections.

6.2 The cost function and the gradient bottleneck

For training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ and a per-example loss $\ell(y, \hat{\mathbf{y}})$ (squared loss for regression, cross-entropy for classification), the **empirical risk** is

$$L(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, f(\mathbf{x}_i; \boldsymbol{\theta})) = \frac{1}{n} \sum_{i=1}^n \ell_i(\boldsymbol{\theta}).$$

Two structural assumptions on ℓ keep backpropagation on a per-example footing:

- **Decomposability:** $L = \frac{1}{n} \sum_i \ell_i$, so $\nabla_{\boldsymbol{\theta}} L = \frac{1}{n} \sum_i \nabla_{\boldsymbol{\theta}} \ell_i$. Backprop computes $\nabla_{\boldsymbol{\theta}} \ell_i$ for one example (or a minibatch) at a time.
- **Output-layer dependence:** ℓ_i depends on $\boldsymbol{\theta}$ only through the output activation $\mathbf{a}^{(L)} = f(\mathbf{x}_i; \boldsymbol{\theta})$. The vector $\nabla_{\mathbf{a}^{(L)}} \ell_i$ is therefore a closed-form expression, supplying the initialization for the backward pass.

Both assumptions hold for squared loss and cross-entropy.

Non-convexity. Composition through nonlinear σ destroys convexity: $L(\boldsymbol{\theta})$ has multiple local minima, saddle points, and large flat regions. Global minimization is intractable; training uses first-order iterative methods that find acceptable local minima.

Definition 6.35 (Stochastic gradient descent). At iteration $t = 1, 2, \dots$, sample a minibatch $\mathcal{B}_t \subset \{1, \dots, n\}$ of size B , choose a learning rate $\eta_t > 0$, and update

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla_{\boldsymbol{\theta}} \ell_i(\boldsymbol{\theta}_t).$$

Under uniform sampling, the minibatch gradient is unbiased for $\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_t)$. Adam, AdaGrad, RMSProp reweight the update direction with running statistics of past gradients; each still calls $\nabla_{\boldsymbol{\theta}} \ell_i$ once per minibatch element.

The gradient bottleneck. Each iteration computes $\nabla_{\boldsymbol{\theta}} \ell$, a vector of length $|\boldsymbol{\theta}| = \sum_{l=1}^L (d_l d_{l-1} + d_l)$. For a modest network with $d_0 = 100$, $d_1 = d_2 = 256$, $d_3 = 10$ ($L = 3$), $|\boldsymbol{\theta}| \approx 9 \cdot 10^4$; production deep networks have $|\boldsymbol{\theta}|$ in the millions to billions. Naive finite differencing computes one component of $\nabla_{\boldsymbol{\theta}} \ell$ per forward pass:

$$\frac{\partial \ell}{\partial \theta_k} \approx \frac{\ell(\boldsymbol{\theta} + h \mathbf{e}_k) - \ell(\boldsymbol{\theta})}{h},$$

costing $O(|\boldsymbol{\theta}| \cdot \text{cost}(f))$ for the entire gradient. At any practical scale this is prohibitive. The algorithm of the next subsection achieves $O(\text{cost}(f))$ for the full gradient, independent of $|\boldsymbol{\theta}|$.

6.3 Computational graphs and reverse-mode automatic differentiation

Reverse-mode automatic differentiation (AD) computes $\nabla_{\mathbf{x}} y$ for any program $y = f(\mathbf{x})$ at cost proportional to evaluating f . It needs two ingredients: a directed acyclic graph (DAG) of elementary

operations (the **computational graph**), and a backward sweep that accumulates partial derivatives along it.

Definition 6.36 (Computational graph). A program that computes $y = f(\mathbf{x})$ via elementary operations (arithmetic, exp, log, sin, matrix-vector product, etc.) induces a directed acyclic graph (DAG) with

- source nodes $w_1, \dots, w_p = x_1, \dots, x_p$ (the inputs);
- internal nodes $w_k = \phi_k(\{w_a\}_{a \in \text{pa}(k)})$ for $k > p$, where $\text{pa}(k) \subset \{1, \dots, k-1\}$ are the parents of w_k and ϕ_k is an elementary operation;
- one sink node identified with the scalar output y .

Example 6.2 (Computational graph of $z = x_1x_2 + \sin x_1$). The program "compute $z = x_1x_2 + \sin x_1$ " decomposes into five intermediate nodes:

$$w_1 = x_1, \quad w_2 = x_2, \quad w_3 = w_1w_2, \quad w_4 = \sin w_1, \quad w_5 = w_3 + w_4 = z.$$

The parents are $\text{pa}(3) = \{1, 2\}$, $\text{pa}(4) = \{1\}$, $\text{pa}(5) = \{3, 4\}$. The local Jacobian entries are $\partial w_3 / \partial w_1 = w_2$, $\partial w_3 / \partial w_2 = w_1$, $\partial w_4 / \partial w_1 = \cos w_1$, $\partial w_5 / \partial w_3 = 1$, $\partial w_5 / \partial w_4 = 1$.

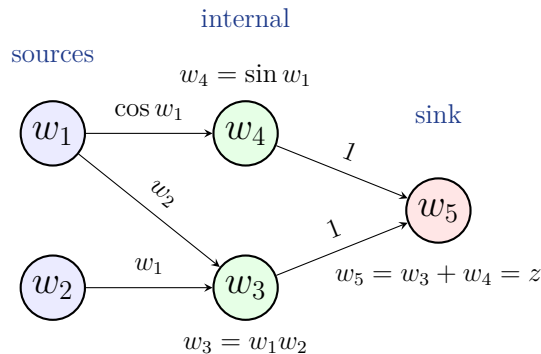


Figure 6.2: Computational graph of $z = x_1 x_2 + \sin x_1$. Sources $w_1 = x_1$, $w_2 = x_2$ on the left; internal nodes $w_3 = w_1 w_2$, $w_4 = \sin w_1$ in the middle; sink $w_5 = z$ on the right. Each edge $w_a \rightarrow w_b$ is labelled by its local Jacobian $\partial w_b / \partial w_a$, evaluated at the cached forward values.

Multivariate chain rule on a DAG. Let w_k be a node and w_a a strict ancestor in the DAG. The partial $\partial w_k / \partial w_a$ accumulates contributions along every path $w_a \rightarrow w_k$:

$$\frac{\partial w_k}{\partial w_a} = \sum_{\text{path } w_a \rightarrow w_k} \prod_{(w_p, w_q) \in \text{path}} \frac{\partial w_q}{\partial w_p}. \quad (*)$$

Proof of the path-sum identity (*)

Induct on k . **Base.** If $w_a \in \text{pa}(k)$, the only path from w_a to w_k is the direct edge (w_a, w_k) , and $\partial w_k / \partial w_a = \partial \phi_k / \partial w_a$, agreeing with the path-sum (a single one-edge term).

Inductive step. Apply the multivariate chain rule at $w_k = \phi_k(\{w_p\}_{p \in \text{pa}(k)})$:

$$\frac{\partial w_k}{\partial w_a} = \sum_{p \in \text{pa}(k)} \frac{\partial \phi_k}{\partial w_p} \cdot \frac{\partial w_p}{\partial w_a}.$$

Each parent w_p has topological depth strictly less than w_k , so by the inductive hypothesis $\partial w_p / \partial w_a$ equals its own path sum over $w_a \rightarrow w_p$. Multiplying by $\partial \phi_k / \partial w_p = \partial w_k / \partial w_p$ extends each path $w_a \rightarrow w_p$ by the edge (w_p, w_k) , producing every path $w_a \rightarrow w_k$ exactly once when summed over $p \in \text{pa}(k)$. ■

The path-sum is algorithmically catastrophic: a graph of depth D with constant fan-in has $O(2^D)$ paths between two nodes, and direct enumeration scales exponentially. Both modes of AD avoid this by reusing intermediate computations.

Forward mode. Sweep the DAG in topological order (parents before children), maintaining alongside each value w_k a **tangent** $\dot{w}_k = \partial w_k / \partial x_j$ for one fixed source index j . Initialize $\dot{w}_a = \mathbb{1}\{a = j\}$ for $a = 1, \dots, p$ and propagate at each internal node by one application of the chain rule:

$$\dot{w}_k = \sum_{p \in \text{pa}(k)} \frac{\partial \phi_k}{\partial w_p} \dot{w}_p.$$

After the sweep, $\dot{y} = \partial y / \partial x_j$. Computing the full gradient $\nabla_{\mathbf{x}} y$ takes p separate sweeps, total cost $O(p \cdot \text{cost}(f))$.

Reverse mode. Two sweeps. **Forward sweep:** compute and cache w_k at every node. **Backward sweep:** sweep in reverse topological order (children before parents), maintaining an **adjoint** $\bar{w}_k = \partial y / \partial w_k$. Initialize $\bar{y} = 1$ at the output sink; propagate at each internal node k by accumulating contributions from every child:

$$\bar{w}_k = \sum_{c: k \in \text{pa}(c)} \frac{\partial \phi_c}{\partial w_k} \bar{w}_c. \quad (\dagger)$$

After the sweep, $\bar{w}_j = \partial y / \partial x_j$ for every source j at once. The two passes together cost $O(\text{cost}(f))$, independent of p .

Why the reverse propagation rule (†) holds

Fix an internal or source node w_k . By the multivariate chain rule, y depends on w_k only through the children w_c of w_k (the nodes with $k \in \text{pa}(c)$); summing the chain rule over those children,

$$\bar{w}_k = \frac{\partial y}{\partial w_k} = \sum_{c: k \in \text{pa}(c)} \frac{\partial y}{\partial w_c} \cdot \frac{\partial w_c}{\partial w_k} = \sum_{c: k \in \text{pa}(c)} \bar{w}_c \cdot \frac{\partial \phi_c}{\partial w_k}.$$

Reverse topological order ensures every $\bar{w}_c$ has already been computed when $\bar{w}_k$ is updated. ■

Example 6.3 (Reverse mode on $z = x_1 x_2 + \sin x_1$ at $\mathbf{x} = (1.5, 0.5)$). **Forward sweep.** Cache the values

$$w_1 = 1.5, \quad w_2 = 0.5, \quad w_3 = w_1 w_2 = 0.75, \quad w_4 = \sin w_1 \approx 0.9975, \quad w_5 = w_3 + w_4 \approx 1.7475.$$

Backward sweep. Initialize $\bar{w}_5 = \partial z / \partial w_5 = 1$, then process nodes in reverse:

$$\bar{w}_4 = \frac{\partial w_5}{\partial w_4} \bar{w}_5 = 1 \cdot 1 = 1,$$

$$\bar{w}_3 = \frac{\partial w_5}{\partial w_3} \bar{w}_5 = 1 \cdot 1 = 1,$$

$$\bar{w}_2 = \frac{\partial w_3}{\partial w_2} \bar{w}_3 = w_1 \cdot 1 = 1.5,$$

$$\bar{w}_1 = \frac{\partial w_3}{\partial w_1} \bar{w}_3 + \frac{\partial w_4}{\partial w_1} \bar{w}_4 = w_2 \cdot 1 + \cos w_1 \cdot 1 = 0.5 + \cos 1.5 \approx 0.5707.$$

For $\bar{w}_1$ both children of w_1 contribute (namely w_3 and w_4); for $\bar{w}_2$ only the child w_3 contributes. Hence $\partial z / \partial x_1 = \bar{w}_1 \approx 0.5707$ and $\partial z / \partial x_2 = \bar{w}_2 = 1.5$, both recovered in one backward sweep.

Forward mode for comparison. Forward mode would require two separate sweeps, one for each input. With $p = 2$ the cost difference is small, but for the parameter vector of a neural network $p = |\boldsymbol{\theta}|$ can be 10^7 , where forward mode is infeasible.

Mode trade-off. For $f : \mathbb{R}^p \rightarrow \mathbb{R}^m$, the Jacobian $\partial f / \partial \mathbf{x}$ is $m \times p$. Forward mode delivers one column per sweep; reverse mode delivers one row per sweep.

Mode	Cost for full Jacobian	Best for
Forward	$O(p \cdot \text{cost}(f))$	$p \ll m$ (few inputs, many outputs)
Reverse	$O(m \cdot \text{cost}(f))$	$m \ll p$ (many inputs, few outputs)

For a neural-network loss, $m = 1$ (the scalar ℓ) and $p = |\boldsymbol{\theta}|$ is in the millions; reverse mode is cheaper than forward by the factor $|\boldsymbol{\theta}|$. This is the cheap-gradient principle: $\nabla_{\boldsymbol{\theta}} \ell$ is computable at the cost of ℓ itself, regardless of $|\boldsymbol{\theta}|$.

6.4 The four backpropagation equations

Reverse-mode AD applied to the MLP recursion of Definition 6.33 yields four named equations that together compute every component of $\nabla_{\boldsymbol{\theta}} \ell$. Each equation is the chain rule applied to one specific dependence, with careful index tracking; the only structural ingredient is the Hadamard identity $\text{diag}(\sigma'(\mathbf{z})) \mathbf{u} = \sigma'(\mathbf{z}) \odot \mathbf{u}$.

Definition 6.37 (Layer error). For each layer $l = 1, \dots, L$, the **error** at layer l is the gradient of ℓ with respect to the pre-activation:

$$\delta_j^{(l)} = \frac{\partial \ell}{\partial z_j^{(l)}}, \quad j = 1, \dots, d_l, \quad \boldsymbol{\delta}^{(l)} \in \mathbb{R}^{d_l}.$$

The error $\delta_j^{(l)}$ measures the sensitivity of the loss to a small perturbation of $z_j^{(l)}$ alone, with all other $z_{j'}^{(l)}$ held fixed. Once the errors are known at every layer, the gradients with respect to weights and biases follow by direct chain rule, since $\mathbf{z}^{(l)}$ depends linearly on $\mathbf{W}^{(l)}, \mathbf{b}^{(l)}, \mathbf{a}^{(l-1)}$.

Theorem 6.38 (Backpropagation: BP1, output-layer error). $\boldsymbol{\delta}^{(L)} = \nabla_{\mathbf{a}^{(L)}} \ell \odot \sigma'(\mathbf{z}^{(L)})$, where $\nabla_{\mathbf{a}^{(L)}} \ell$ is the gradient of ℓ with respect to the output activation.

Proof of BP1

By the chain rule applied componentwise,

$$\delta_j^{(L)} = \frac{\partial \ell}{\partial z_j^{(L)}} = \sum_{k=1}^{d_L} \frac{\partial \ell}{\partial a_k^{(L)}} \cdot \frac{\partial a_k^{(L)}}{\partial z_j^{(L)}}.$$

The activation is componentwise, so $a_k^{(L)} = \sigma(z_k^{(L)})$ depends on $z_j^{(L)}$ only when $k = j$:

$$\frac{\partial a_k^{(L)}}{\partial z_j^{(L)}} = \sigma'(z_j^{(L)}) \mathbb{1}\{k = j\}.$$

Substituting collapses the sum to its $k = j$ term,

$$\delta_j^{(L)} = \frac{\partial \ell}{\partial a_j^{(L)}} \cdot \sigma'(z_j^{(L)}),$$

which is the j -th component of $\nabla_{\mathbf{a}^{(L)}} \ell \odot \sigma'(\mathbf{z}^{(L)})$. ■

For squared loss with linear output, $\nabla_{\mathbf{a}^{(L)}} \ell = \mathbf{a}^{(L)} - \mathbf{y}$ and $\sigma' \equiv 1$, so $\boldsymbol{\delta}^{(L)} = \mathbf{a}^{(L)} - \mathbf{y}$. For cross-entropy with softmax output, the same expression collapses to $\boldsymbol{\delta}^{(L)} = \text{softmax}(\mathbf{z}^{(L)}) - \mathbf{y}$ for one-hot $\mathbf{y}$ (combining the softmax Jacobian with the cross-entropy gradient).

Theorem 6.39 (Backpropagation: BP2, error recursion). For $l = 1, \dots, L - 1$,

$$\boldsymbol{\delta}^{(l)} = (\mathbf{W}^{(l+1)})^\top \boldsymbol{\delta}^{(l+1)} \odot \sigma'(\mathbf{z}^{(l)}).$$

Proof of BP2

By the chain rule, ℓ depends on $z_j^{(l)}$ through every component of the next pre-activation $\mathbf{z}^{(l+1)}$:

$$\delta_j^{(l)} = \frac{\partial \ell}{\partial z_j^{(l)}} = \sum_{k=1}^{d_{l+1}} \frac{\partial \ell}{\partial z_k^{(l+1)}} \cdot \frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}} = \sum_{k=1}^{d_{l+1}} \delta_k^{(l+1)} \cdot \frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}}.$$

The pre-activation at layer $l + 1$ expands as

$$z_k^{(l+1)} = \sum_{i=1}^{d_l} W_{ki}^{(l+1)} a_i^{(l)} + b_k^{(l+1)} = \sum_{i=1}^{d_l} W_{ki}^{(l+1)} \sigma(z_i^{(l)}) + b_k^{(l+1)}.$$

Differentiating with respect to $z_j^{(l)}$, only the $i = j$ term in the sum contributes (the other i 's produce $\sigma'(z_i^{(l)}) \cdot 0 = 0$):

$$\frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}} = W_{kj}^{(l+1)} \sigma'(z_j^{(l)}).$$

Substituting into the chain-rule sum,

$$\delta_j^{(l)} = \sum_{k=1}^{d_{l+1}} \delta_k^{(l+1)} W_{kj}^{(l+1)} \sigma'(z_j^{(l)}) = \left(\sum_{k=1}^{d_{l+1}} W_{kj}^{(l+1)} \delta_k^{(l+1)} \right) \sigma'(z_j^{(l)}).$$

The factor $\sigma'(z_j^{(l)})$ does not depend on k and is pulled out of the sum. The inner sum is the

j -th component of the matrix-vector product $\mathbf{W}^{(l+1)\top} \boldsymbol{\delta}^{(l+1)}$ (the transpose puts $W_{kj}^{(l+1)}$ in row j column k). Stacking over $j = 1, \dots, d_l$ gives the vector form. ■

Theorem 6.40 (Backpropagation: BP3, bias gradient). For $l = 1, \dots, L$, $\frac{\partial \ell}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)}$.

Proof of BP3

By the chain rule, ℓ depends on $b_j^{(l)}$ only through $\mathbf{z}^{(l)}$:

$$\frac{\partial \ell}{\partial b_j^{(l)}} = \sum_{k=1}^{d_l} \frac{\partial \ell}{\partial z_k^{(l)}} \cdot \frac{\partial z_k^{(l)}}{\partial b_j^{(l)}}.$$

From $z_k^{(l)} = \sum_i W_{ki}^{(l)} a_i^{(l-1)} + b_k^{(l)}$, the only $b^{(l)}$ -dependence in $z_k^{(l)}$ is the additive term $b_k^{(l)}$, so $\partial z_k^{(l)} / \partial b_j^{(l)} = \mathbf{1}\{k = j\}$. Only the $k = j$ term survives, giving $\partial \ell / \partial b_j^{(l)} = \delta_j^{(l)}$. ■

Theorem 6.41 (Backpropagation: BP4, weight gradient). For $l = 1, \dots, L$,

$$\frac{\partial \ell}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)} \mathbf{a}^{(l-1)\top}, \quad \text{i.e.,} \quad \frac{\partial \ell}{\partial W_{jk}^{(l)}} = \delta_j^{(l)} a_k^{(l-1)}.$$

Proof of BP4

By the chain rule, ℓ depends on $W_{jk}^{(l)}$ only through $\mathbf{z}^{(l)}$:

$$\frac{\partial \ell}{\partial W_{jk}^{(l)}} = \sum_{i=1}^{d_l} \frac{\partial \ell}{\partial z_i^{(l)}} \cdot \frac{\partial z_i^{(l)}}{\partial W_{jk}^{(l)}}.$$

From $z_i^{(l)} = \sum_m W_{im}^{(l)} a_m^{(l-1)} + b_i^{(l)}$, the entry $W_{jk}^{(l)}$ appears only in $z_j^{(l)}$ with coefficient $a_k^{(l-1)}$:

$$\frac{\partial z_i^{(l)}}{\partial W_{jk}^{(l)}} = \mathbb{1}\{i = j\} a_k^{(l-1)}.$$

Only $i = j$ survives, giving $\partial\ell/\partial W_{jk}^{(l)} = \delta_j^{(l)} a_k^{(l-1)}$. The (j, k) entries form the rank-one matrix $\boldsymbol{\delta}^{(l)} \mathbf{a}^{(l-1)\top}$. ■

Reading the equations. BP1 anchors the recursion at the output: the layer- L error is the loss gradient against the activation, multiplied componentwise by σ' at the layer- L pre-activation. BP2 propagates the error backward: the next-layer error is multiplied by $\mathbf{W}^{(l+1)\top}$ (the transpose of the forward weights), then componentwise by $\sigma'(\mathbf{z}^{(l)})$. BP3 says the bias gradient at layer l equals the error at layer l (one-to-one). BP4 says the weight gradient is the outer product of the layer's error with the previous layer's activation. Each equation is the chain rule with one specific local Jacobian made explicit.

Numerical illustration. Apply BP1–BP4 to a 2-2-1 sigmoid MLP with squared loss and explicit parameters. The R chunk below evaluates one forward pass, then computes each $\boldsymbol{\delta}^{(l)}$ and gradient via the four equations, with the BP-equation tag inline.

```

1  sigmoid <- function(z) 1 / (1 + exp(-z))
2  dsigmoid <- function(z) sigmoid(z) * (1 - sigmoid(z))
3
4  # 2 inputs -> 2 hidden -> 1 output, sigmoid throughout, squared loss
5  W1 <- matrix(c(0.5, -0.3,
6                0.2,  0.8), 2, 2, byrow = TRUE);  b1 <- c(0.1, -0.2)
7  W2 <- matrix(c(0.4, -0.6), 1, 2);  b2 <- 0
8  x  <- c(1, 2);  y <- 1
9
10 # Forward pass (Definition of MLP)
11 z1 <- W1 %*% x + b1;  a1 <- sigmoid(z1)
12 z2 <- W2 %*% a1 + b2;  a2 <- sigmoid(z2)
13
14 # Backward via BP1-BP4
15 delta2 <- (a2 - y) * dsigmoid(z2)  # BP1
16 dW2     <- delta2 %*% t(a1)          # BP4 at l = 2
17 db2     <- as.vector(delta2)         # BP3 at l = 2
18 delta1 <- (t(W2) %*% delta2) * dsigmoid(z1)  # BP2
19 dW1     <- delta1 %*% t(x)           # BP4 at l = 1
20 db1     <- as.vector(delta1)         # BP3 at l = 1

```

```
21  
22 list(loss = round(0.5 * drop((a2 - y)^2), 4),  
23      delta1 = round(drop(delta1), 4), delta2 = round(drop(delta2), 4),  
24      dW1 = round(dW1, 4), db1 = round(db1, 4),  
25      dW2 = round(dW2, 4), db2 = round(db2, 4))
```

```
## $loss  
## [1] 0.1649  
##  
## $delta1  
## [1] -0.0140 0.0118  
##  
## $delta2  
## [1] -0.1404  
##  
## $dW1  
##      [,1] [,2]  
## [1,] -0.0140 -0.0281  
## [2,] 0.0118 0.0235  
##  
## $db1
```

```
## [1] -0.0140  0.0118
##
## $dW2
##          [,1]      [,2]
## [1,] -0.0702 -0.1168
##
## $db2
## [1] -0.1404
```

The same six lines (BP1, BP2 once each; BP3 and BP4 at each layer) compute every component of $\nabla_{\theta}\ell$ at a cost dominated by two matrix products per layer.

6.5 The backpropagation algorithm

Combining BP1–BP4 with one forward pass and one backward pass yields the full algorithm.

Algorithm: Backpropagation for one example $(\mathbf{x}, \mathbf{y})$

1. Set $\mathbf{a}^{(0)} = \mathbf{x}$.
 2. For $l = 1, \dots, L$: $\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$, $\mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)})$.
 3. Compute $\boldsymbol{\delta}^{(L)} = \nabla_{\mathbf{a}^{(L)}} \ell \odot \sigma'(\mathbf{z}^{(L)})$ (BP1).
 4. For $l = L, L-1, \dots, 1$:

$\partial \ell / \partial \mathbf{W}^{(l)} = \boldsymbol{\delta}^{(l)} \mathbf{a}^{(l-1)\top}$ (BP4).
 $\partial \ell / \partial \mathbf{b}^{(l)} = \boldsymbol{\delta}^{(l)}$ (BP3).
 If $l > 1$: $\boldsymbol{\delta}^{(l-1)} = (\mathbf{W}^{(l)\top} \boldsymbol{\delta}^{(l)}) \odot \sigma'(\mathbf{z}^{(l-1)})$ (BP2).
-

Cost. Forward and backward at layer l each cost $O(d_l d_{l-1})$ (the dominant operations $\mathbf{W}^{(l)} \mathbf{a}^{(l-1)}$ and $\mathbf{W}^{(l)\top} \boldsymbol{\delta}^{(l)}$ have matching shapes), so the full gradient $\nabla_{\boldsymbol{\theta}} \ell$ over $|\boldsymbol{\theta}| = \sum_l (d_l d_{l-1} + d_l)$ parameters costs twice the forward pass, regardless of $|\boldsymbol{\theta}|$.

Minibatch. Vectorize by stacking B inputs columnwise as $\mathbf{X} \in \mathbb{R}^{p \times B}$ and propagating $\mathbf{A}^{(l)}, \mathbf{Z}^{(l)}, \Delta^{(l)} \in \mathbb{R}^{d_l \times B}$. Forward and backward become dense matrix-matrix products of cost $O(d_l d_{l-1} B)$ per layer; the summed weight gradient is $\sum_{i=1}^B \delta_i^{(l)} \mathbf{a}_i^{(l-1)\top} = \Delta^{(l)} \mathbf{A}^{(l-1)\top}$.

6.6 R implementation from scratch

The full algorithm, specialized to a 2-layer MLP ($L = 2$, ReLU hidden + sigmoid output, squared loss), fits in roughly thirty lines of base R. The architecture matches modern practice: ReLU on hidden layers (the standard since deep learning's revival), sigmoid on the output for binary classification (mapping to $(0, 1)$). The implementation below trains the network on the binary XOR pattern, a classification problem no linear classifier solves.

Activations and derivatives. Two activations: ReLU on the hidden layer, sigmoid on the output. The form $(z > 0) * z$ preserves matrix structure that `pmax(0, z)` would flatten; the convention $\text{ReLU}'(0) := 0$ is standard for subgradient implementations.

```
1 relu      <- function(z) (z > 0) * z
2 drelu     <- function(z) (z > 0) * 1
3 sigmoid   <- function(z) 1 / (1 + exp(-z))
```

```
4 dsigmoid <- function(z) sigmoid(z) * (1 - sigmoid(z))
```

Parameter initialization. Hidden-layer weights use He scaling $\mathcal{N}(0, 2/d_{l-1})$: the variance multiplier 2 compensates for ReLU zeroing out the negative half, keeping $\text{Var}(\mathbf{a}^{(l)})$ roughly constant across layers. The sigmoid output layer uses Glorot scaling $\mathcal{N}(0, 1/d_{l-1})$. Biases start at zero.

```
1 init_mlp <- function(d_in, d_hidden, d_out) {  
2   list(  
3     W1 = matrix(rnorm(d_hidden * d_in,    sd = sqrt(2 / d_in)),    d_hidden, d_in),  
4     b1 = numeric(d_hidden),  
5     W2 = matrix(rnorm(d_out * d_hidden, sd = 1 / sqrt(d_hidden)), d_out,    d_hidden),  
6     b2 = numeric(d_out)  
7   )  
8 }
```

Forward pass. Stack B training examples columnwise as $\mathbf{X} \in \mathbb{R}^{d_{\text{in}} \times B}$. The call `sweep(M, 1, v, "+")` adds the length- d_l vector v to every column of M ; this is more explicit than R's recycling

rule and remains correct when $B = 1$.

```

1 forward <- function(par, X) {
2   Z1 <- sweep(par$W1 %*% X, 1, par$b1, "+")
3   A1 <- relu(Z1)
4   Z2 <- sweep(par$W2 %*% A1, 1, par$b2, "+")
5   A2 <- sigmoid(Z2)
6   list(Z1 = Z1, A1 = A1, Z2 = Z2, A2 = A2)
7 }

```

Backward pass. Each line implements one of BP1–BP4 in matrix form. BP1 contracts the output error against the sigmoid derivative; BP2 backprops through the ReLU derivative. `rowSums(as.matrix(...))` sums the per-example bias gradients across the minibatch; the `as.matrix` wrap forces a matrix shape when the output layer is one-dimensional ($d_{\text{out}} = 1$ would otherwise yield a length- B vector that `rowSums` mishandles).

```

1 backward <- function(par, cache, X, Y) {
2   D2 <- (cache$A2 - Y) * dsigmoid(cache$Z2)           # BP1: sigmoid output error
3   dW2 <- D2 %*% t(cache$A1)                           # BP4 at l = 2

```

```

4 db2 <- rowSums(as.matrix(D2))           # BP3 at l = 2
5 D1  <- (t(par$W2) %*% D2) * drelu(cache$Z1) # BP2: ReLU hidden error
6 dW1 <- D1 %*% t(X)                     # BP4 at l = 1
7 db1 <- rowSums(as.matrix(D1))           # BP3 at l = 1
8 list(dW1 = dW1, db1 = db1, dW2 = dW2, db2 = db2, B = ncol(X))
9 }

```

Full-batch gradient step.

```

1 sgd_step <- function(par, X, Y, eta) {
2   cache <- forward(par, X)
3   grad  <- backward(par, cache, X, Y)
4   par$W1 <- par$W1 - eta * grad$dW1 / grad$B
5   par$b1 <- par$b1 - eta * grad$db1 / grad$B
6   par$W2 <- par$W2 - eta * grad$dW2 / grad$B
7   par$b2 <- par$b2 - eta * grad$db2 / grad$B
8   par
9 }

```

The XOR pattern. XOR is the canonical example of a binary classification problem unsolvable by any linear classifier yet solvable by a 2-layer MLP.

Definition 6.42 (XOR pattern). The **XOR pattern** is the binary classification problem on the four points $\mathbf{x}_1 = (0, 0)$, $\mathbf{x}_2 = (0, 1)$, $\mathbf{x}_3 = (1, 0)$, $\mathbf{x}_4 = (1, 1)$ in $\{0, 1\}^2$ with labels $y_1 = -1$, $y_2 = +1$, $y_3 = +1$, $y_4 = -1$ (the output is $+1$ exactly when one input is 0 and the other is 1).

Theorem 6.43 (XOR is not linearly separable). There is no $(\mathbf{w}, b) \in \mathbb{R}^2 \times \mathbb{R}$ for which the linear-classifier rule $\hat{y}(\mathbf{x}_i) = \text{sign}(\mathbf{w}^\top \mathbf{x}_i + b)$ matches y_i at all four points of Definition 6.42.

Proof of Theorem 6.43

Suppose such $(\mathbf{w}, b)$ exists. Correct classification at every point requires $y_i(\mathbf{w}^\top \mathbf{x}_i + b) > 0$:

- $\mathbf{x}_1 = (0, 0)$, $y_1 = -1$: $-b > 0$, so $b < 0$;
- $\mathbf{x}_4 = (1, 1)$, $y_4 = -1$: $-(w_1 + w_2 + b) > 0$, i.e., $w_1 + w_2 < -b$;
- $\mathbf{x}_2 = (0, 1)$, $y_2 = +1$: $w_2 + b > 0$, i.e., $w_2 > -b$;
- $\mathbf{x}_3 = (1, 0)$, $y_3 = +1$: $w_1 + b > 0$, i.e., $w_1 > -b$.

Adding the last two strict inequalities, $w_1 + w_2 > -2b$. Combining with the second, $-2b < w_1 + w_2 < -b$, so $-2b < -b$, hence $b > 0$. This contradicts $b < 0$. ■

A linear classifier (logistic regression, linear SVM, perceptron) thus fails on XOR; a 2-layer MLP suffices. The next example exhibits a 2-layer ReLU network that computes XOR exactly with hand-chosen integer weights, before the SGD-trained version below.

Example 6.4 (A 2-layer ReLU MLP that computes XOR exactly). The 2-2-1 MLP with ReLU activation on the hidden layer and identity on the output, with weights and biases

$$\mathbf{W}^{(1)} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \quad \mathbf{b}^{(1)} = \begin{pmatrix} 0 \\ -1 \end{pmatrix}, \quad \mathbf{W}^{(2)} = \begin{pmatrix} 2 & -4 \end{pmatrix}, \quad b^{(2)} = -1,$$

returns $f(\mathbf{x}_i; \boldsymbol{\theta}) = y_i$ exactly at every point of Definition 6.42.

$\mathbf{x}$	$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$	$\mathbf{a}^{(1)} = \text{ReLU}(\mathbf{z}^{(1)})$	$\mathbf{z}^{(2)} = \mathbf{W}^{(2)}\mathbf{a}^{(1)} + b^{(2)}$	$f(\mathbf{x})$	y
(0, 0)	(0, -1)	(0, 0)	$2 \cdot 0 + (-4) \cdot 0 - 1 = -1$	-1	-1
(0, 1)	(1, 0)	(1, 0)	$2 \cdot 1 + (-4) \cdot 0 - 1 = +1$	+1	+1
(1, 0)	(1, 0)	(1, 0)	$2 \cdot 1 + (-4) \cdot 0 - 1 = +1$	+1	+1
(1, 1)	(2, 1)	(2, 1)	$2 \cdot 2 + (-4) \cdot 1 - 1 = -1$	-1	-1

The two hidden units extract Boolean features: $a_1^{(1)} = \text{ReLU}(x_1 + x_2)$ counts the number of inputs that are on, while $a_2^{(1)} = \text{ReLU}(x_1 + x_2 - 1)$ fires only when both inputs are on. The output combines them as $2a_1^{(1)} - 4a_2^{(1)} - 1$, giving $-1, +1, +1, -1$ on the four XOR points. Theorem 6.43 forbids any single linear unit from doing this; one hidden layer of two ReLU neurons suffices because their composition realizes the nonlinear feature “exactly one of the inputs is on.”

Demo: learning XOR. The hand-constructed weights of Example 6.4 solve XOR exactly with ReLU hidden + identity output. The demo below keeps the ReLU hidden layer and switches to sigmoid output, whose range $(0, 1)$ matches the rescaled labels $\tilde{y} = (1 + y)/2 \in \{0, 1\}$ for binary classification; stochastic gradient descent discovers its own weights from random initialization, with

four training examples, one hidden layer of four neurons, and 5000 full-batch updates at learning rate $\eta = 0.5$.

```
1 X_xor <- matrix(c(0, 0, 0, 1, 1, 0, 1, 1), nrow = 2)
2 Y_xor <- matrix(c(0, 1, 1, 0), nrow = 1)
3
4 set.seed(42)
5 par <- init_mlp(d_in = 2, d_hidden = 4, d_out = 1)
6 for (epoch in 1:5000) par <- sgd_step(par, X_xor, Y_xor, eta = 0.5)
7 round(forward(par, X_xor)$A2, 3)
```

```
##      [,1] [,2] [,3] [,4]
## [1,] 0.022 0.987 0.987 0.01
```

The output approximates (0, 1, 1, 0). The same six functions, with deeper architectures and mini-batch loops added, scale to networks with millions of parameters; the role of backpropagation is unchanged.

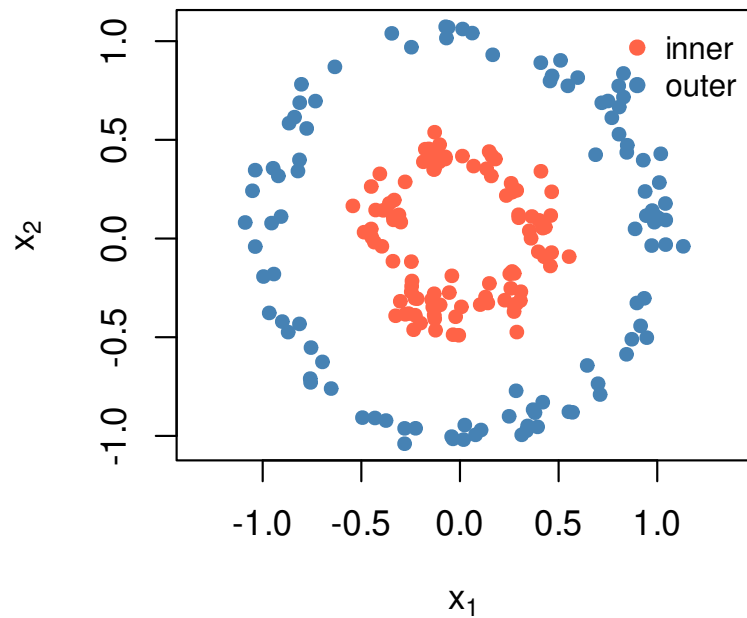
7 Implementation Tour

The theory of §§2–6 maps onto three R packages: **kernlab** (solver-direct SVM), **tidymodels** (workflow + tuning), and **brulee** (MLP via a torch backend). The tour runs all three on a nonlinearly-separable 2D dataset and compares decision boundaries against tree and boosting baselines from Unit 5.

Benchmark data. A two-class concentric-circles dataset: 100 points uniformly distributed on each of two circles in $\mathbb{R}^2$ (radii 0.4 and 1.0, with isotropic Gaussian noise). The classes are not linearly separable; an SVM with RBF kernel separates them by an annular boundary.

```
1 set.seed(2024)
2 n <- 100
3 theta <- runif(2 * n, 0, 2 * pi)
4 r      <- c(rep(0.4, n), rep(1.0, n)) + rnorm(2 * n, sd = 0.07)
5 df_xy <- data.frame(
6   x1 = r * cos(theta),
7   x2 = r * sin(theta),
8   y  = factor(rep(c("inner", "outer"), each = n))
9 )
10
```

```
11 plot(x2 ~ x1, data = df_xy,  
12      col = ifelse(y == "inner", "tomato", "steelblue"),  
13      pch = 19, cex = 0.8, asp = 1,  
14      xlab = expression(x[1]), ylab = expression(x[2]))  
15 legend("topright", legend = levels(df_xy$y),  
16      col = c("tomato", "steelblue"), pch = 19, cex = 0.9, bty = "n")
```



7.1 SVM via kernlab

`kernlab::ksvm` fits the dual problem of §5.4 (Theorem 6.22). The two essential arguments are `kernel` (one of `vanilladot`, `rbfdot`, `polydot`, ...) and `C` (soft-margin); for RBF, `kpar = list(sigma =  $\gamma$ )` fixes the locality.

```

1 library(kernlab)
2 fit_lin <- ksvm(y ~ x1 + x2, data = df_xy, type = "C-svc",
3               kernel = "vanilladot", kpar = list(),
4               C = 1, scaled = TRUE)
5 fit_rbf <- ksvm(y ~ x1 + x2, data = df_xy, type = "C-svc",
6               kernel = "rbfdot", kpar = list(sigma = 1),
7               C = 1, scaled = TRUE)
8
9 train_err <- function(fit) mean(predict(fit, df_xy) != df_xy$y)
10 data.frame(kernel = c("linear", "RBF"),
11            trn_err = c(train_err(fit_lin), train_err(fit_rbf)),
12            num_SV = c(nSV(fit_lin), nSV(fit_rbf)))

```

```

##   kernel trn_err num_SV
## 1 linear   0.375   186

```

```
| ## 2      RBF    0.000      26
```

The linear kernel cannot separate concentric rings; its training error stays substantial and over 90% of points become support vectors (a soft-margin SVM on linearly inseparable data has $\lambda_i^* = C$ for many i , the bound-SV regime of §4.2). The RBF kernel separates the rings cleanly with a much smaller support set.

7.2 SVM via tidymodels with tuning

tidymodels separates model specification (`parsnip`), preprocessing recipe, and resampling-based hyperparameter search. For an RBF SVM with `cost` and `rbf_sigma` tuned via 5-fold CV:

```
1 library(tidymodels)
2
3 split <- initial_split(df_xy, prop = 0.8, strata = y)
4 train <- training(split); test <- testing(split)
5
6 rec <- recipe(y ~ x1 + x2, data = train) |>
7   step_normalize(all_numeric_predictors())
8
9 spec <- svm_rbf(cost = tune(), rbf_sigma = tune()) |>
```

```
10  set_engine("kernlab") |>
11  set_mode("classification")
12
13  wf    <- workflow() |> add_recipe(rec) |> add_model(spec)
14  folds <- vfold_cv(train, v = 5, strata = y)
15  grid  <- grid_regular(cost(), rbf_sigma(), levels = 4)
16
17  tuned <- tune_grid(wf, resamples = folds, grid = grid,
18                    metrics = metric_set(accuracy, roc_auc))
19  best  <- select_best(tuned, metric = "roc_auc")
20
21  final <- finalize_workflow(wf, best) |> last_fit(split)
22  collect_metrics(final)
```

```
## # A tibble: 3 x 4
##   .metric      .estimator .estimate .config
##   <chr>        <chr>         <dbl> <chr>
## 1 accuracy    binary           1     pre0_mod0_post0
## 2 roc_auc     binary           1     pre0_mod0_post0
## 3 brier_class binary       0.0237 pre0_mod0_post0
```

The recipe `step_normalize` applies z -score scaling using each fold's training statistics during CV, avoiding the leakage flagged in Unit 4 §4.6.3. `select_best` picks the (`cost`, `rbf_sigma`) combination with highest CV AUC; `last_fit` retrains on the full training split and evaluates once on the held-out test split.

7.3 MLP via parsnip

A multilayer perceptron via parsnip's `brulee` engine: parsnip routes the specification through a torch backend and trains by minibatch SGD with momentum, standard in modern deep-learning libraries.

```
1 library(brulee)
2 torch::torch_manual_seed(123)
3
4 spec_mlp <- mlp(hidden_units = 16, epochs = 300, learn_rate = 0.05) |>
5   set_engine("brulee",
6             optimizer = "SGD",
7             batch_size = 16,
8             momentum = 0.9) |>
9   set_mode("classification")
10
11 mlp_fit <- workflow() |>
```



```
12 add_recipe(rec) |>
13 add_model(spec_mlp) |>
14 last_fit(split)
15
16 collect_metrics(mlp_fit)
```

```
## # A tibble: 3 x 4
##   .metric      .estimator .estimate .config
##   <chr>        <chr>         <dbl> <chr>
## 1 accuracy    binary           1     pre0_mod0_post0
## 2 roc_auc     binary           1     pre0_mod0_post0
## 3 brier_class binary       0.000171 pre0_mod0_post0
```

Sixteen hidden ReLU units, batch size 16, momentum 0.9, learning rate 0.05, 300 epochs. The recipe-defined standardization is applied per fold, avoiding leakage. `brulee`'s torch backend exposes standard training hooks: early stopping (`stop_iter`), learning-rate schedules (`rate_schedule`), weight decay (`penalty`), and dropout. Deeper architectures use the `brulee_two_layer` engine for two hidden layers, or call `brulee::brulee_mlp(hidden_units = c(64, 32, 16), ...)` directly for arbitrary depth.

BFGS, L-BFGS, and SGD

Quasi-Newton methods like BFGS (Broyden–Fletcher–Goldfarb–Shanno) carry an approximation $\mathbf{B}_k$ to the Hessian $\nabla^2 \ell(\boldsymbol{\theta}_k)$, updated by a rank-two formula from the parameter step $\mathbf{s}_k = \boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_k$ and the gradient change $\mathbf{y}_k = \nabla \ell(\boldsymbol{\theta}_{k+1}) - \nabla \ell(\boldsymbol{\theta}_k)$, then step

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha_k \mathbf{B}_k^{-1} \nabla \ell(\boldsymbol{\theta}_k),$$

with α_k chosen by line search. Storing $\mathbf{B}_k$ takes $O(|\boldsymbol{\theta}|^2)$ memory, prohibitive once $|\boldsymbol{\theta}|$ exceeds $\sim 10^4$. The limited-memory variant L-BFGS reconstructs $\mathbf{B}_k^{-1} \nabla \ell$ from the most recent m pairs $(\mathbf{s}_j, \mathbf{y}_j)$ at $O(m|\boldsymbol{\theta}|)$ memory, and is brulee's default `optimizer = "LBFGS"`. Modern deep-learning training instead uses minibatch SGD (Definition 6.35) and adaptive variants like Adam, both at $O(|\boldsymbol{\theta}|)$ memory and parallelizable across GPU minibatches. The two regimes split by problem size: BFGS-family methods minimize iteration count on small smooth problems; SGD-family methods scale by trading per-iteration progress for cheap iterations on large architectures.

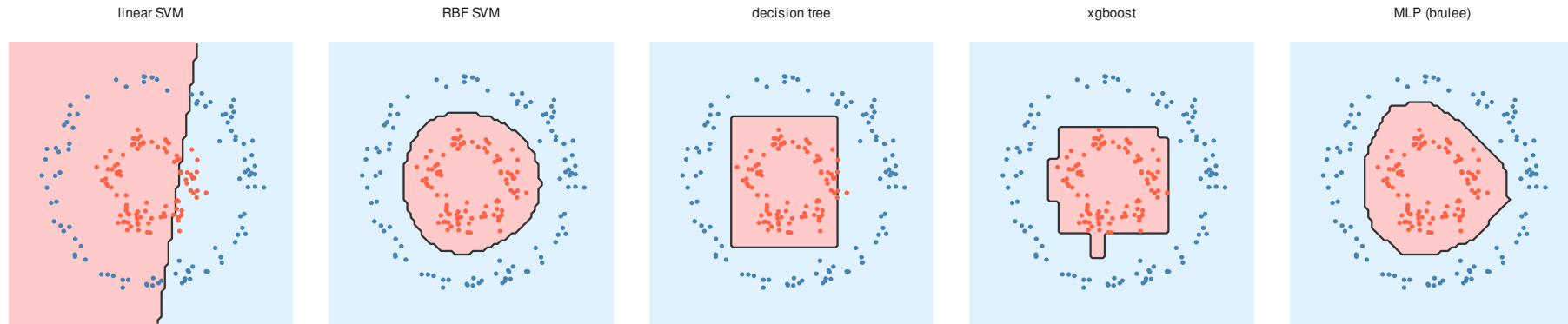
7.4 Decision-boundary comparison

Five models, overlaid on an 80×80 grid: linear and RBF SVM (§§2–5), decision tree and gradient boosting (Unit 5), MLP (§6). The visualization exposes each model's inductive bias.

```
1 library(rpart); library(xgboost); library(brulee); library(ggplot2)
2 library(parsnip); library(dplyr); library(tidyr)
3
4 gx <- seq(-1.4, 1.4, length.out = 80)
5 gy <- seq(-1.4, 1.4, length.out = 80)
6 grid <- expand.grid(x1 = gx, x2 = gy)
7
8 fit_l <- ksvm(y ~ x1 + x2, df_xy, kernel = "vanilladot",
9             kpar = list(), C = 1)
10 fit_r <- ksvm(y ~ x1 + x2, df_xy, kernel = "rbfdot",
11             kpar = list(sigma = 1), C = 1)
12 fit_t <- rpart(y ~ x1 + x2, data = df_xy,
13              control = rpart.control(cp = 0.005))
14 mat <- as.matrix(df_xy[, c("x1", "x2")])
15 dtrain <- xgb.DMatrix(data = mat, label = as.numeric(df_xy$y) - 1)
16 fit_xgb <- xgb.train(
17   params = list(objective = "binary:logistic", max_depth = 3, eta = 0.3),
```

```
18 data      = dtrain, nrounds = 100, verbose = 0)
19 torch::torch_manual_seed(123)
20 spec_mlp <- mlp(hidden_units = 16, epochs = 300, learn_rate = 0.05) |>
21   set_engine("brulee",
22             optimizer = "SGD",
23             batch_size = 16,
24             momentum   = 0.9) |>
25   set_mode("classification")
26 fit_nn <- spec_mlp |> fit(y ~ x1 + x2, data = df_xy)
27
28 is_outer <- function(p) as.integer(p == "outer")
29 preds <- list(
30   `linear SVM`      = is_outer(predict(fit_l, grid)),
31   `RBF SVM`        = is_outer(predict(fit_r, grid)),
32   `decision tree`   = is_outer(predict(fit_t, grid, type = "class")),
33   `xgboost`         = as.integer(predict(fit_xgb, as.matrix(grid)) > 0.5),
34   `MLP (brulee)`    = is_outer(predict(fit_nn, grid)$pred_class)
35 )
36
37 grid_long <- bind_cols(grid, as_tibble(preds)) |>
38   pivot_longer(cols = -c(x1, x2), names_to = "model", values_to = "pred") |>
```

```
39 mutate(model = factor(model, levels = names(preds)))
40
41 ggplot(grid_long, aes(x1, x2)) +
42   geom_tile(aes(fill = factor(pred))) +
43   geom_contour(aes(z = pred), breaks = 0.5,
44               color = "grey20", linewidth = 0.5) +
45   geom_point(data = df_xy, aes(color = y), size = 0.6) +
46   scale_fill_manual(values = c(`0` = "#FECACA", `1` = "#E0F2FE"),
47                    guide = "none") +
48   scale_color_manual(values = c(inner = "tomato", outer = "steelblue"),
49                      guide = "none") +
50   facet_wrap(~ model, nrow = 1) +
51   coord_fixed() +
52   theme_minimal(base_size = 9) +
53   theme(panel.grid = element_blank(),
54         axis.title = element_blank(),
55         axis.text = element_blank(),
56         axis.ticks = element_blank())
```



The five panels reveal the inductive biases:

- linear SVM commits to a hyperplane and cannot represent the annulus;
- RBF SVM produces a near-circular boundary, with locality controlled by γ ;
- the decision tree partitions the plane into axis-aligned rectangles, missing the rotational symmetry;
- xgboost stacks many shallow trees, approximating the annulus by overlapping rectangles;
- the MLP composes affine + ReLU layers and produces a piecewise-linear nonlinear boundary;

with sufficient hidden units, it approximates the annulus comparably to RBF SVM.

On data whose true boundary is rotationally symmetric, kernel and MLP methods adapt; tree-based models adapt only via increased depth and ensemble size.

8 Pitfalls and Misapplications

The pitfalls below arise from common misuses of SVM and neural-network methods. Each can be embedded as a trap in a final-project dataset.

8.1 Forgetting to standardize

The SVM dual depends on the data only through inner products or kernel values derived from them. Both are scale-sensitive: a feature with magnitude 1000 swamps one with magnitude 1 in the inner product, and the resulting decision boundary effectively ignores the small-magnitude feature.

Why tempting. Tree-based methods (Unit 5) are scale-invariant under monotone feature transformations; students may carry that habit over to SVM and skip standardization.

```
1 set.seed(7)
2 n <- 80
```

```
3 df <- data.frame(  
4   x1 = c(rnorm(n, -1, 0.7), rnorm(n, 1, 0.7)),  
5   x2 = c(rnorm(n, -1, 0.7), rnorm(n, 1, 0.7)) * 1000,  
6   y  = factor(c(rep("A", n), rep("B", n)))  
7 )  
8  
9 fit_err <- function(scaled) {  
10   fit <- ksvm(y ~ x1 + x2, data = df, type = "C-svc",  
11             kernel = "rbfdot", scaled = scaled, C = 1)  
12   mean(predict(fit, df) != df$y)  
13 }  
14 c(unscaled = fit_err(scaled = FALSE),  
15   scaled    = fit_err(scaled = TRUE))
```

```
## unscaled    scaled  
## 0.08750     0.01875
```

Diagnostic and fix. Inspect $\text{sd}(\mathbf{x}_j)$ across features; standardize via z-score before fitting an SVM. `kernlab::ksvm` sets `scaled = TRUE` by default; the failure mode requires explicit `scaled = FALSE` or an inconsistent train/test scaling.

8.2 Gamma–cost interaction in RBF kernels

The RBF kernel has two hyperparameters that interact: γ (locality) and the soft-margin C . Large γ + large C overfits (each support vector forms its own basin). Small γ + small C underfits (the kernel is near-constant; C tolerates many violations).

Why tempting. Tuning either alone misses the joint optimum; one-dimensional grid searches at fixed γ produce misleading results.

```
1 set.seed(2025)
2 n <- 100
3 theta <- runif(2 * n, 0, 2 * pi)
4 r      <- c(rep(0.4, n), rep(1.0, n)) + rnorm(2 * n, sd = 0.07)
5 df <- data.frame(x1 = r * cos(theta), x2 = r * sin(theta),
6                  y   = factor(rep(c("A", "B"), each = n)))
7
8 fit_at <- function(C, sigma) {
9   fit <- ksvm(y ~ x1 + x2, data = df, kernel = "rbfdot",
10              kpar = list(sigma = sigma), C = C)
11   c(C = C, sigma = sigma,
```

```

12   train_err = mean(predict(fit, df) != df$y), nSV = nSV(fit))
13 }
14 rbind(fit_at(1,    0.1),    # mild gamma, mild C    : appropriate
15       fit_at(1,    100),    # large gamma, mild C   : memorizes points
16       fit_at(0.01, 1),     # mild gamma, tiny C    : underfits
17       fit_at(100,  1))     # mild gamma, huge C     : near-hard-margin

```

```

##           C sigma train_err nSV
## [1,] 1e+00   0.1         0  66
## [2,] 1e+00 100.0         0 182
## [3,] 1e-02   1.0         0 200
## [4,] 1e+02   1.0         0  18

```

Fix. Joint grid search over (γ, C) via `tune::tune_grid`; report the full surface, not the marginal best.

8.3 Class imbalance

With class imbalance (90:10, say), the default soft-margin SVM uses one C for both classes, so the hyperplane shifts toward the minority to minimize total loss. Overall accuracy is high; minority recall

is low.

Why tempting. Reporting accuracy alone hides the failure; a model predicting "majority" everywhere already scores 90%.

```
1 set.seed(7)
2 n_maj <- 180; n_min <- 20
3 df <- data.frame(
4   x1 = c(rnorm(n_maj, 0, 1), rnorm(n_min, 1.2, 0.3)),
5   x2 = c(rnorm(n_maj, 0, 1), rnorm(n_min, 1.2, 0.3)),
6   y = factor(c(rep("majority", n_maj), rep("minority", n_min)))
7 )
8
9 fit_d <- ksvm(y ~ x1 + x2, data = df, type = "C-svc",
10             kernel = "rbfdot", C = 1)
11 fit_w <- ksvm(y ~ x1 + x2, data = df, type = "C-svc",
12             kernel = "rbfdot", C = 1,
13             class.weights = c(majority = 1, minority = n_maj / n_min))
14
15 eval_minrec <- function(fit) {
```

```
16 p <- predict(fit, df)
17 c(accuracy      = mean(p == df$y),
18     minority_recall = mean(p[df$y == "minority"] == "minority"))
19 }
20 rbind(default = eval_minrec(fit_d),
21       balanced = eval_minrec(fit_w))
```

```
##          accuracy minority_recall
## default      0.960             0.75
## balanced      0.905             1.00
```

Fix. Pass `class.weights` (kernlab) or `case_weights` (tidymodels) inversely proportional to class frequency. SMOTE inside CV folds is the alternative if upsampling is desired (Unit 3 §3.7.4).

8.4 Neural networks on small data

A neural network with $|\boldsymbol{\theta}|$ exceeding n can memorize the training set, leading to overfitting. On small tabular data (n in the hundreds to low thousands), tree ensembles and kernel SVMs typically match or outperform MLPs in test accuracy.

Why tempting. Recent NN successes on images and language give MLPs a default-modern-method reputation; students may apply them where they don't fit.

```
1 set.seed(1)
2 n <- 80
3 df <- data.frame(x1 = rnorm(n), x2 = rnorm(n))
4 df$y <- factor(ifelse(df$x1 + df$x2 + rnorm(n, 0, 0.4) > 0, "A", "B"))
5 i <- sample(n, n * 0.7)
6
7 fit_rp <- rpart(y ~ ., data = df[i, ], method = "class")
8
9 torch::torch_manual_seed(1)
10 spec_mlp <- mlp(hidden_units = 32, epochs = 200, learn_rate = 0.05) |>
11   set_engine("brulee",
12             optimizer = "SGD",
13             batch_size = 8,
14             momentum = 0.9) |>
15   set_mode("classification")
16 fit_nn <- spec_mlp |> fit(y ~ ., data = df[i, ])
17
18 err_class <- function(fit, dat) mean(predict(fit, dat, type = "class") != dat$y)
```

```
19 err_parsnip <- function(fit, dat) mean(predict(fit, dat)$pred_class != dat$y)
20 data.frame(
21   method    = c("decision tree", "MLP (32 hidden)"),
22   test_err   = c(err_class(fit_rp, df[-i, ]),
23                 err_parsnip(fit_nn, df[-i, ]))
24 )
```

```
##           method    test_err
## 1  decision tree 0.08333333
## 2  MLP (32 hidden) 0.08333333
```

Fix. Compare a baseline (logistic regression, kernel SVM, gradient boosting) to the NN; if the NN does not significantly beat the baseline at the available n , prefer the simpler, more interpretable method.

8.5 No universally best kernel

Kernel choice encodes assumed similarity structure: linear assumes flat boundaries; polynomial assumes interactions up to a fixed degree; RBF assumes smooth and locally-constant. No kernel dominates across all problems.

Why tempting. The RBF default in many libraries gives reasonable performance on smooth low-dimensional data, encouraging always-RBF thinking.

```
1 set.seed(11); n <- 100
2
3 A <- data.frame(x1 = rnorm(2 * n), x2 = rnorm(2 * n))
4 A$y <- factor(ifelse(A$x1 + A$x2 + rnorm(2 * n, 0, 0.3) > 0, "A1", "A2"))
5
6 B <- data.frame(x1 = runif(2 * n, -2, 2), x2 = runif(2 * n, -2, 2))
7 B$y <- factor(ifelse(B$x1 * B$x2 + rnorm(2 * n, 0, 0.2) > 0, "B1", "B2"))
8
9 theta <- runif(2 * n, 0, 2 * pi)
10 C_df <- data.frame(x1 = c(0.4, 1)[rep(1:2, each = n)] * cos(theta),
11                   x2 = c(0.4, 1)[rep(1:2, each = n)] * sin(theta),
12                   y = factor(rep(c("C1", "C2"), each = n)))
13
14 err_at <- function(d, kern, ...) {
15   fit <- ksvm(y ~ x1 + x2, data = d, type = "C-svc", kernel = kern, ..., C = 1)
16   round(mean(predict(fit, d) != d$y), 3)
17 }
18
```

```

19 rbind(
20   `linear data`      = c(linear = err_at(A, "vanilladot", kpar = list()),
21                          poly2  = err_at(A, "polydot",    kpar = list(degree = 2)),
22                          rbf     = err_at(A, "rbfdot",     kpar = list(sigma = 1))),
23   `polynomial data` = c(linear = err_at(B, "vanilladot", kpar = list()),
24                          poly2  = err_at(B, "polydot",    kpar = list(degree = 2)),
25                          rbf     = err_at(B, "rbfdot",     kpar = list(sigma = 1))),
26   `rings data`      = c(linear = err_at(C_df, "vanilladot", kpar = list()),
27                          poly2  = err_at(C_df, "polydot",    kpar = list(degree = 2)),
28                          rbf     = err_at(C_df, "rbfdot",     kpar = list(sigma = 1)))
29 )

```

```

##           linear poly2   rbf
## linear data    0.060 0.065 0.065
## polynomial data 0.400 0.095 0.075
## rings data     0.395 0.000 0.000

```

Linear wins narrowly on A but fails on B and C ; both polynomial and RBF handle the nonlinear data, with relative quality differing by dataset. No kernel dominates across all three.

Fix. Cross-validate over kernel choices; do not assume RBF is always best.

8.6 SVM probabilities and Platt scaling

The SVM decision function $f^*(\mathbf{x}) = \sum_{i \in \mathcal{S}} \lambda_i^* y_i K(\mathbf{x}_i, \mathbf{x}) + b^*$ is a real-valued score, not a probability. Calibrated probabilities require **Platt scaling**: a logistic regression $p(y = 1 \mid f^*) = \sigma(af^* + b)$ fit on a held-out validation set. Calibrating on the training data overfits and produces over-confident probabilities.

Why tempting. A naive "probability" via $\sigma(\text{decision})$ on the training data looks well-defined but skips the calibration step.

```
1 set.seed(1)
2 n <- 200
3 df <- data.frame(
4   x1 = c(rnorm(n, 0, 1), rnorm(n, 1.5, 1)),
5   x2 = c(rnorm(n, 0, 1), rnorm(n, 1.5, 1)),
6   y  = factor(c(rep("A", n), rep("B", n)))
7 )
8 i_train <- sample(2 * n, n)
```

```
9 train <- df[ i_train, ]; test <- df[-i_train, ]
10
11 fit_naive <- ksvm(y ~ x1 + x2, data = train, type = "C-svc",
12                 kernel = "rbfdot", C = 1, prob.model = FALSE)
13 fit_platt <- ksvm(y ~ x1 + x2, data = train, type = "C-svc",
14                 kernel = "rbfdot", C = 1, prob.model = TRUE)
15
16 dec <- predict(fit_naive, test, type = "decision")
17 p_nv <- plogis(as.numeric(dec))
18 p_cal <- predict(fit_platt, test, type = "probabilities")[, "B"]
19
20 y_t <- as.integer(test$y == "B")
21 data.frame(
22   scheme = c("plogis(decision)", "Platt CV (prob.model=TRUE)"),
23   brier = round(c(mean((p_nv - y_t)^2), mean((p_cal - y_t)^2)), 4)
24 )
```

```
##               scheme brier
## 1          plogis(decision) 0.138
## 2 Platt CV (prob.model=TRUE) 0.117
```

Fix. Use `prob.model = TRUE` (kernlab) or an explicit calibration step (`cal_estimate_logistic` / `cal_estimate_isotonic` in tidymodels). Report Brier score or reliability diagrams to verify calibration.

References

Bernhard Schölkopf and Alexander J. Smola. **Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond**. MIT Press, Cambridge, MA, 2002.